```
        Model 172 (Skyhawk) </label>
        <br />
        <label> <input type = "radio"  name = "planeButton"
                       value = "182"
                       onclick = "planeChoice(182)" />
        Model 182 (Skylane) </label>
        <br />
        <label> <input type = "radio"  name = "planeButton"
                       value = "210"
                       onclick = "planeChoice(210)" />
        Model 210 (Centurian) </label>
      </p>
    </form>
  </body>
</html>
```

```
// radio_click.js
//   An example of the use of the click event with radio buttons,
//   registering the event handler by assignment to the button
//   attributes

// The event handler for a radio button collection
function planeChoice (plane) {

// Produce an alert message about the chosen airplane
  switch (plane) {
    case 152:
      alert("A small two-place airplane for flight training");
      break;
    case 172:
      alert("The smaller of two four-place airplanes");
      break;
    case 182:
      alert("The larger of two four-place airplanes");
      break;
    case 210:
      alert("A six-place high-performance airplane");
      break;
    default:
      alert("Error in JavaScript function planeChoice");
      break;
  }
}
```

Figure 5.3 shows a browser display of `radio_click.html`. Figure 5.4 shows the `alert` window that results from choosing the Model 182 radio button in `radio_click.html`.

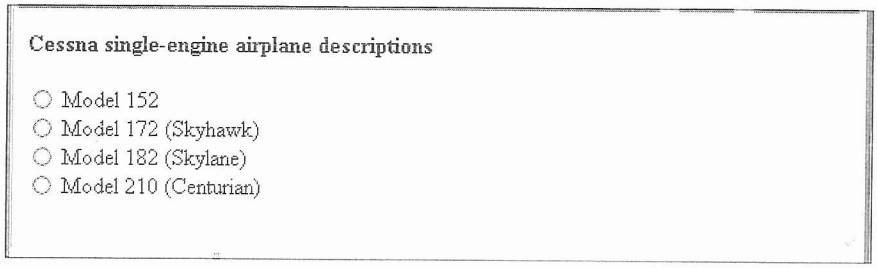

Figure 5.3 Display of `radio_click.html`

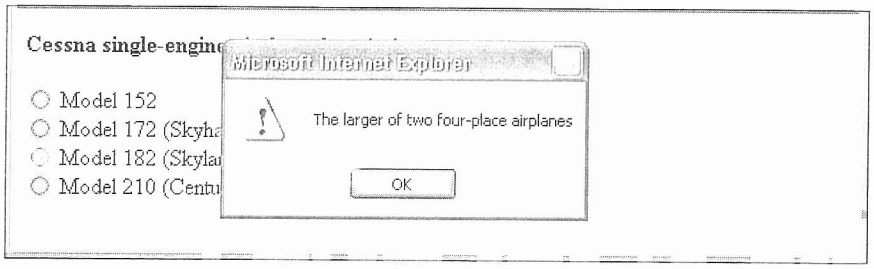

Figure 5.4 The result of pressing the Model 182 button in `radio_click`

In `radio_click.html` the event handler is registered by assigning its call to the `onclick` attribute of the radio buttons. The specific button that was clicked is identified by the parameter sent in the handler call in the button element. An alternative to using the parameter would be to include code in the handler to determine which radio button was pressed.

The next example, `radio_click2.html`, whose purpose is the same as that of `radio_click.html`, registers the event handler by assigning the name of the handler to the event properties of the radio button objects. For example, the following registers the handler on the first radio button:

```
document.getElementById("myForm").elements[0].onclick = planeChoice;
```

Recall that this statement must follow both the handler function and the XHTML form specification so that JavaScript has seen both before assigning to the property. The following example uses three files, one for the XHTML, one for the script for the event handlers, and one for the script to register the handlers.

```
<!-- radio_click2.hmtl
     A document for radio_click2.js
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> radio_click2.html </title>

<!-- Script for the event handler -->
  <script type = "text/javascript"  src = "radio_click2.js" >
  </script>

  </head>
  <body>
    <h4> Cessna single-engine airplane descriptions </h4>
    <form id = "myForm"  action = "">
      <p>
        <label> <input type = "radio"  name = "planeButton"
                       value = "152" />
        Model 152 </label>
        <br />
        <label> <input type = "radio"  name = "planeButton"
                       value = "172" />
        Model 172 (Skyhawk) </label>
        <br />
        <label> <input type = "radio"  name = "planeButton"
                       value = "182" />
        Model 182 (Skylane) </label>
        <br />
        <label> <input type = "radio"  name = "planeButton"
                       value = "210" />
        Model 210 (Centurian) </label>
      </p>
    </form>

<!-- Script for registering the event handlers -->
    <script type = "text/javascript"  src = "radio_click2r.js" >
    </script>

  </body>
</html>
```

```
// radio_click2.js
//   An example of the use of the click event with radio buttons,
//   registering the event handler by assigning an event property

// The event handler for a radio button collection
function planeChoice (plane) {

// Put the DOM address of the elements array in a local variable
  var dom = document.getElementById("myForm");

// Determine which button was pressed
  for (var index = 0; index < dom.planeButton.length;
       index++) {
    if (dom.planeButton[index].checked) {
      plane = dom.planeButton[index].value;
      break;
  }
}

// Produce an alert message about the chosen airplane
  switch (plane) {
    case "152":
      alert("A small two-place airplane for flight training");
      break;
    case "172":
      alert("The smaller of two four-place airplanes");
      break;
    case "182":
      alert("The larger of two four-place airplanes");
      break;
    case "210":
      alert("A six-place high-performance airplane");
      break;
    default:
      alert("Error in JavaScript function planeChoice");
      break;
  }
}
```

```
// radio_click2r.js
//   The event registering code for radio_click2
var dom = document.getElementById("myForm");
dom.elements[0].onclick = planeChoice;
dom.elements[1].onclick = planeChoice;
dom.elements[2].onclick = planeChoice;
dom.elements[3].onclick = planeChoice;
```

In `radio_click2r.js` (the JavaScript file that registers the event handlers), the form elements (radio buttons in this case) are addressed as elements of the `elements` array. An alternative would be to give each radio button an `id` attribute and register the handler using the `id`. For example, the first radio button could be defined as follows:

```
<input type = "radio"  name = "planeButton" value = "152"
   id = "152" />
```

Then the event handler registration would be as follows:

```
var dom = document.getElementById("myForm");
dom.getElementById("152").onclick = planeChoice;
dom.getElementById("172").onclick = planeChoice;
dom.getElementById("182").onclick = planeChoice;
dom.getElementById("210").onclick = planeChoice;
```

There is no way to specify parameters on the handler function when it is registered by assigning its name to the event property. Therefore, event handlers that are registered this way cannot use parameters—clearly a disadvantage of this approach. In `radio_click2.js`, the handler includes a loop to determine which radio button created the `click` event.

There are two advantages to registering handlers as properties over registering them in XHTML attributes. First, it is good to keep XHTML and JavaScript separated in the document. This allows a kind of modularization of XHTML documents, resulting in a cleaner design that will be easier to maintain. Second, having the handler function registered as the value of a property allows for the possibility of changing it during use. This could be done by registering a different handler for the event when some other event occurred. This would be impossible if the handler were registered using XHTML.

5.7 Handling Events from Text Box and Password Elements

Text boxes and passwords can create four different events: `blur`, `focus`, `change`, and `select`.

5.7.1 The Focus Event

Suppose JavaScript is used to precompute the total cost of an order and display it to the customer before the order is submitted to the server for processing. An unscrupulous user may be tempted to change the total cost before submission, thinking that somehow an altered (and lower) price would not be noticed at the server end. Such a change to a text box can be prevented by an event handler that blurs the text box every time the user attempts to put it in focus. Blur can be forced on an element with the `blur` method. The following example illustrates this process:

```
<?xml version = "1.0"  encoding = "utf-8" ?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- nochange.html
     A document for nochange.js
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> nochange.html </title>

<!-- Script for the event handlers -->
    <script type = "text/javascript"  src = "nochange.js" >
    </script>

  </head>
  <body>
    <form action = "">
      <h3> Coffee Order Form </h3>

<!-- A bordered table for item orders -->
      <table border = "border">

<!-- First, the column headings -->
        <tr>
```

```
          <th> Product Name </th>
          <th> Price </th>
          <th> Quantity </th>
        </tr>

<!-- Now, the table data entries -->
        <tr>
          <th> French Vanilla (1 lb.) </th>
          <td> $3.49 </td>
          <td> <input type = "text"  id = "french"
                      size ="2" /> </td>
        </tr>
        <tr>
          <th> Hazlenut Cream (1 lb.) </th>
          <td> $3.95 </td>
          <td> <input type = "text"  id = "hazlenut"
                size = "2" /> </td>
        </tr>
        <tr>
          <th> Colombian (1 lb.) </th>
          <td> $4.59 </td>
          <td> <input type = "text"  id = "colombian"
                size = "2" /></td>
        </tr>
      </table>

<!-- Button for precomputation of the total cost -->
      <p>
        <input type = "button"  value = "Total Cost"
               onclick = "computeCost();" />
        <input type = "text"  size = "5"  id = "cost"
               onfocus = "this.blur();" />
      </p>

<!-- The submit and reset buttons -->
      <p>
        <input type = "submit"  value = "Submit Order" />
        <input type = "reset"  value = "Clear Order Form" />
      </p>
    </form>
  </body>
</html>
```

```
// nochange.js
//   This script illustrates using the focus event
//   to prevent the user from changing a text field

// The event handler function to compute the cost
function computeCost() {
  var french = document.getElementById("french").value;
  var hazlenut = document.getElementById("hazlenut").value;
  var columbian = document.getElementById("colombian").value;

// Compute the cost
  document.getElementById("cost").value =
  totalCost = french * 3.49 + hazlenut * 3.95 +
              colombian * 4.59;
}  //* end of computeCost
```

In this example, the button labeled `Total Cost` allows the user to precompute the total cost of the order. The event handler for this button gets the values (input quantities) of the three kinds of coffee and computes the total cost. The cost value is placed in the text box's value property, and it is then displayed for the user. Whenever this text box acquires focus, it is forced to blur with the `blur` method, which prevents the user from changing the value.

5.7.2 Validating Form Input

As stated earlier, checking the format and completeness of form input is a common application of JavaScript. This approach shifts this task from the usually busy server to the client, which in most cases is only lightly used. It also results in less network traffic because it avoids sending bad data to the server, only to have it returned without being processed. Furthermore, validating form data on the client results in quicker responses to users.

When a user fills in a form input element incorrectly and a JavaScript event-handler function detects the error, the function should do several things. First, it should produce an `alert` message indicating the error to the user and specifying the correct format of the input. Next, it should cause the input element to be put in focus, which positions the cursor in the element. This is done with the `focus` method, which must be called through the DOM address of the element. For example, if the element's `id` is `phone`, the element can be put in focus with the following statement:

```
document.getElementById("phone").focus();
```

This puts the cursor in the `phone` text box. Finally, the function should select the element, which highlights the text in the element. This is done with the `select` method, as in the following example:

```
document.getElementById("phone").select();
```

If an event handler returns `false`, that tells the browser not to perform any default actions of the event. For example, if the event is a click on the *Submit* button, the default action is to submit the form data to the server for processing. If user input is being validated in an event handler that is called when the `submit` event occurs and some of the input is incorrect, the handler should return `false` to avoid sending the bad data to the server. We use the convention that event handlers that check form data always return `false` if they detect an error, and `true` otherwise.

When a form requests a password from the user and that password will be used in future sessions, the user is often asked to enter the password a second time for verification. A JavaScript function can be used to check that the two entered passwords are the same.

The form in the following example includes the two password input elements, along with *Reset* and *Submit* buttons. The JavaScript function that checks the passwords is called either when the *Submit* button is pressed, using the `onsubmit` event to trigger the call, or when the second text box loses focus, using the `blur` event. The function performs two different tests. First, it determines whether the user typed the initial password (in the first input box) by testing the value of the element against the empty string. If no password has been typed into the first field, the function calls `alert` to produce an error message, calls `focus` on the field, and returns `false`. The second test is to determine whether the two typed passwords are the same. If they are different, the function calls `alert` to generate an error message, calls both `focus` and `select` on the first password field, and returns `false`. If they are the same, it returns `true`. Following is the XHTML document that creates the text boxes for the passwords, as well as the *Reset* and *Submit* buttons, and the two scripts for the event handlers for `pswd_chk.html` and the event handler registrations.

```
<?xml version = "1.0" encoding = "utf-8" ?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- pswd_chk.html
     A document for pswd_chk.ps
     Creates two text boxes for passwords
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Illustrate password checking> </title>
    <script type = "text/javascript"  src = "pswd_chk.js" >
```

```
    </script>
  </head>
  <body>
    <h3> Password Input </h3>
    <form id = "myForm"  action = "" >
      <p>

      <label> Your password
        <input type = "password" id = "initial"
               size = "10" />
      </label>
      <br /><br />

      <label> Verify password
        <input type = "password"  id = "second"
               size = "10" />
      </label>
      <br /><br />

      <input type = "reset"  name = "reset" />
      <input type = "submit"  name = "submit" />
      </p>
    </form>

<!-- Script for registering the event handlers  -->
    <script type = "text/javascript"  src = "pswd_chkr.js">
    </script>

  </body>
</html>
```

```
// pswd_chk.js
//   An example of input password checking, using the submit
//   event

// The event handler function for password checking
function chkPasswords() {
  var init = document.getElementById("initial");
  var sec = document.getElementById("second");
  if (init.value == "") {
    alert("You did not enter a password \n" +
          "Please enter one now");
    init.focus();
    return false;
```

```
  }
  if (init.value != sec.value) {
    alert("The two passwords you entered are not the same \n" +
          "Please re-enter both now");
    init.focus();
    init.select();
    return false;
  } else
    return true;
}
```

```
// pswd_chkr.js
//   Register the event handlers for pswd_chk.html

document.getElementById("second").onblur = chkPasswords;
document.getElementById("myForm").onsubmit = chkPasswords;
```

Figure 5.5 shows a browser display of `pswd_chk.html` after the two password elements have been input but before *Submit Query* has been clicked.

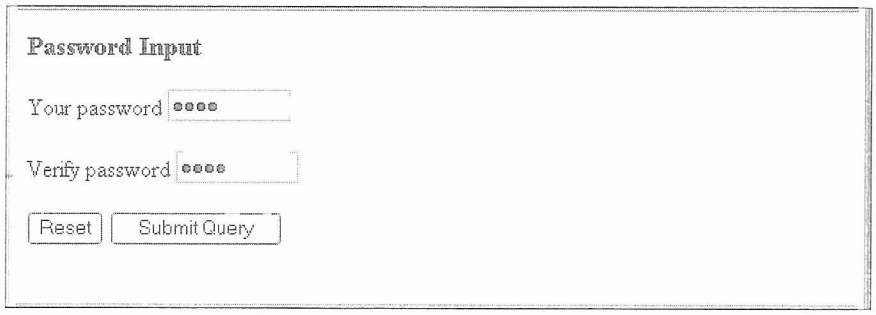

Figure 5.5 Display of `pswd_chk.html` after it has been filled out

Figure 5.6 shows a browser display that results from pressing the *Submit Query* button on `pswd_chk.html` after different passwords have been entered.

Figure 5.6 Display of `pswd_chk.html` after *Submit Query* has been clicked

We now consider an example that checks the validity of the form values for a name and phone number obtained from text widgets. Functions are used to check the form of each input when the values of the text boxes are changed, which is detected by the appearance of a `change` event.

In both cases, if an error is detected, an `alert` message is generated and both `focus` and `select` are called to prompt the user to fix the input. The `alert` message includes the correct format. The correct format for the name is last-name, first-name, middle-initial, where the first and last names must begin with uppercase letters and have at least one lowercase letter. Both must be followed immediately by a comma and possibly one space. The middle initial must be uppercase. It may or may not be followed by a period. There can be no characters before or after the whole name. The pattern for matching such names is as follows:

```
/^[A-Z][a-z]+, ?[A-Z][a-z]+, ?[A-Z]\.?$/
```

Note the use of the anchors, `^` and `$`, on the ends of the pattern. This prevents any leading or trailing characters. Also, notice the question marks after the spaces (following the first and last names) and after the period. Recall that the question mark qualifier means zero or one of the qualified subpattern. The period is backslashed so it matches only a period.

The correct format of the phone number is three digits and a dash, followed by three digits and a dash, followed by four digits. As with names, no characaters can precede or follow the phone number. The pattern for phone numbers is as follows:

```
/^d{3}-\d{3}-\d{4}$/
```

The following is the XHTML document, `validator.html`, that displays the text boxes for a customer's name and phone number.

```
<?xml version = "1.0" encoding = "utf-8" ?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- validator.html
     A document for validator.js
     Creates text boxes for a name and a phone number
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Illustrate form input validation> </title>
    <script type = "text/javascript"  src = "validator.js" >
    </script>
  </head>
  <body>
    <h3> Customer Information </h3>
    <form action = "">
      <p>
        <label>
          <input type = "text"  id = "custName" />
          Name (last name, first name, middle initial)
        </label>
        <br /><br />

        <label>
          <input type = "text"  id = "phone" />
          Phone number (ddd-ddd-dddd)
        </label>
        <br /><br />

        <input type = "reset"  id = "reset" />

        <input type = "submit"  id = "submit" />
      </p>
    </form>

<!-- An inline script for the event handler registrations -->
    <script type = "text/javascript">
      <!--
// Set form element object properties to their
// corresponding event handler functions

        document.getElementById("custName").onchange = chkName;
```

```
        document.getElementById("phone").onchange = chkPhone;
      // -->
    </script>
  </body>
</html>
```

The following is the script for the event handlers for `validator.html`.

```
// validator.js
//   An example of input validation using the change and submit
//   events

// The event handler function for the name text box
function chkName() {
  var myName = document.getElementById("custName");

// Test the format of the input name
//  Allow the spaces after the commas to be optional
//  Allow the period after the initial to be optional
  var pos = myName.value.search(
            /^[A-Z][a-z]+, ?[A-Z][a-z]+, ?[A-Z]\.?$/);
  if (pos != 0) {
    alert("The name you entered (" + myName.value +
          ") is not in the correct form. \n" +
          "The correct form is: " +
          "last-name, first-name, middle-initial \n" +
          "Please go back and fix your name");
    myName.focus();
    myName.select();
    return false;
  } else
    return true;
}

// The event handler function for the phone number text box
function chkPhone() {
  var myPhone = document.getElementById("phone");

// Test the format of the input phone number
  var pos = myPhone.value.search(/^\d{3}-\d{3}-\d{4}$/);
  if (pos != 0) {
    alert("The phone number you entered (" + myPhone.value +
          ") is not in the correct form. \n" +
          "The correct form is: ddd-ddd-dddd \n" +
          "Please go back and fix your phone number");
```

```
    myPhone.focus();
    myPhone.select();
    return false;
  } else
    return true;
}
```

Figure 5.7 shows the browser screen of `validator.html` after entering a name in the correct format, followed by an invalid telephone number. The screen is shown before the user causes the phone text field to lose focus, either by pressing Enter or by clicking the left mouse button outside the phone text field.

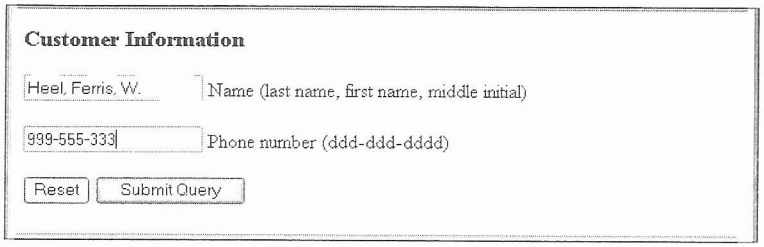

Figure 5.7 Display of `validator.html`, with an invalid phone number, while the phone text field has focus

Figure 5.8 shows the `alert` dialog box generated by pressing the Enter button in the phone text field of the screen of Figure 5.7.

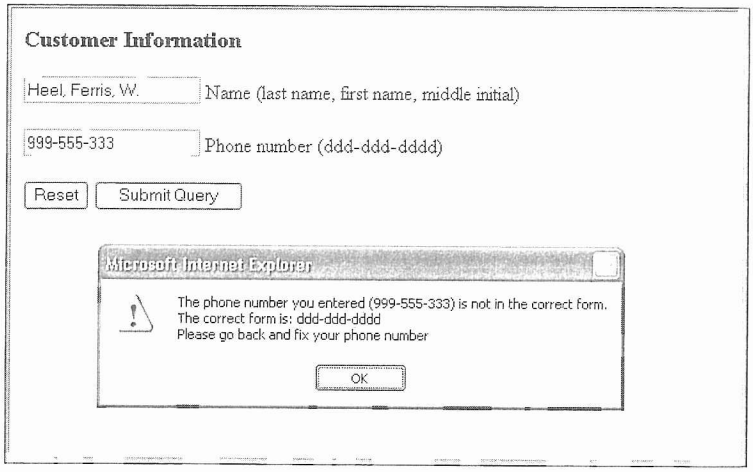

Figure 5.8 The message created by entering an invalid telephone number in `validator.html`

5.8 The DOM 2 Event Model

The DOM 2 event model does not include the features of the DOM 0 event model. However, there is no chance that support for those features will be dropped from browsers anytime soon. Therefore, Web authors should not hesitate to continue to use them. On the other hand, the DOM 2 event model is more sophisticated and powerful than DOM 0. The real drawback of using the DOM 2 model is that Microsoft has yet to provide support for it in its browsers.

The DOM 2 model is a modularized interface. One of the DOM 2 modules is Events, which includes several submodules. The most commonly used are `HTMLEvents` and `MouseEvents`. The interfaces and events defined by these modules are as follows:

Module	Event Interface	Event Types
`HTMLEvents`	`Event`	`abort, blur, change, error, focus, load, reset, resize, scroll, select, submit, unload`
`MouseEvents`	`MouseEvent`	`click, mousedown, mousemove, mouseout, mouseover, mouseup`

When an event occurs and there is an event handler that is called, an object that implements the event interface associated with the event type is implicitly passed to the handler. (Section 5.8.1 explains how a handler is chosen to be called.) The properties of this object have information associated with the event.

The DOM 2 event model is relatively complex. This section covers only the basics of the model. A description of the rest of the model can be found at the W3C's Web site.

5.8.1 Event Propagation

The connection between an event and the handler that deals with it is very simple in the DOM 0 event model. When the browser senses an event has occurred, the object associated with the element that caused the event is checked for event handlers. If that object has a registered handler for the particular event that occurred, that handler is executed. The event-handler connection for the DOM 2 event model is much more complicated.

Briefly, what happens is as follows. An event object is created at a node in the document tree. For that event, that node is called the *target node*. Event creation causes a three-phase process to begin.

The first of these phases is called the *capturing phase*. The event starts at the document root node and propagates down the tree to the target node. If there are any handlers for the event registered on any node encountered in this propagation, including the document node but not the target node, these handlers are checked to determine whether they are enabled. (Section 5.8.2 explains how a handler can be defined to be enabled.) Any enabled handler for the event that is

found during capturing is executed (whether it is enabled or not). When the event reaches the target node, the second phase takes place, in which the handlers registered for the event at the target node are executed. The second phase is similar to what happens with the DOM 0 event model. After execution of any appropriate handlers at the target node, the third phase begins. This is the *bubbling phase*, in which the event bubbles back up the document tree to the document node. On this trip back up the tree, any handler registered for the event at any node on the way is executed (whether it is enabled or not).

Not all events bubble. For example, the `load` and `unload` events do not bubble. On the other hand, all of the mouse events do. In general, if it makes sense to handle an event farther up the document tree than the target node, the event bubbles; otherwise, it does not.

Any handler can stop the event from further propagation, using the `stopPropagation` method of the event object.

Bubbling is an idea that was borrowed from exception handling. In a large and complicated document, having event handlers for every element would require a great deal of code. Much of this code would be redundant, both in the handlers and in the registering of handlers for events. Therefore, it makes sense to define a way for a single handler to deal with events created from a number of similar elements. The concept is that events can be propagated to some central place for handling rather than always being handled locally. In the DOM, the natural central place for event handling is at the document or window level, so that is the direction of bubbling.

Many events cause the browser to perform some action; for example, a mouse click on a link causes the document referenced in the link to replace the current document. In some cases, we want to prevent this action from taking place. For example, if a value in a form is found to be invalid by a *Submit* button event handler, we do not want the form to be submitted to the server. In the DOM 0 event model, the action is prevented by having the handler return `false`. The DOM 2 Events interface provides a method, `preventDefault`, that accomplishes the same thing.

5.8.2 Event Handler Registration

The DOM 0 event model uses two different ways of registering event handlers. First, the handler code can be assigned as a string literal to the event's associated attribute in the element. Second, the name of the handler function can be assigned to the property associated with the event. Handler registration in the DOM 2 event model is performed by the method `addEventListener`, which is defined in the `EventTarget` interface, which is implemented by all objects that descend from `Document`.[2]

2. The name of this method includes "listener" rather than "handler" because in the DOM 2 specification handlers are called *listeners*. This is also the term used in Java for widget event handlers.

The addEventListener method takes three parameters, the first of which is the name of the event as a string literal. For example, "mouseup" and "submit" would be legitimate first parameters. The second parameter is the handler function. This could be specified as the function code itself or as the name of a function that is defined elsewhere. The third parameter is a Boolean value that specifies whether the handler is enabled for calling during the capturing phase. If the value true is specified, the handler is enabled for the capturing phase. In fact, an enabled handler can *only* be called during capturing. If the value is false, the handler can be called either at the target node or on any node reached during bubbling.

When a handler is called, it is passed a single parameter, the event object. For example, suppose we want to register the event handler chkName on the text element whose id is custName for the change event. The following call accomplishes this:

```
document.custName.addEventListener(
                    "change", chkName, false);
```

In this case, we want the handler to be called at the target node, which is custName in this example, so we passed false as the third parameter.

Sometimes it is convenient to have a temporary event handler. This can be done by registering the handler for the time when it is to be used, and then deleting that registration. The removeEventListener method deletes the registration of an event handler. This method takes the same parameters as addEventListener.

With the DOM 0 event model, when an event handler is registered to a document node, the handler becomes a method of the object that represents that node. This makes every use of this in the handler a reference to the target node. FX2 browsers implement event handlers for the DOM 2 model in this same way. However, this is not required by the DOM 2 model, so some other browsers may not use this approach, making the use of this in a handler potentially nonportable. The safe alternative is to use the currentTarget property of Event, which will always reference the object on which the handler is being executed. If the handler is called through the object of the target node, currentTarget is the target node. However, if the handler is called during capturing or bubbling, currentTarget is the object through which the handler is called, which is not the target node object. Another property of Event, target, is a reference to the target node.

The MouseEvent interface inherits from the Event interface. It adds a collection of properties related to mouse events. The most useful of these are clientX and clientY, which have the x and y coordinates of the mouse cursor, relative to the upper-left corner of the client area of the browser window. The whole browser window is taken into account, so if the user has scrolled down the document, the clientY value is measured from the top of the document, not the top of the current display.

5.8.3 An Example of the DOM 2 Event Model

The following example is a revision of the `validator.html` document `validator.js` script from Section 5.7, which used the DOM 0 event model. Because this version uses the DOM 2 event model, it does not work with IE7. Notice that no call to `preventDefault` appears in this document. The only event handled here is `change`, which has no default actions, so there is nothing to prevent.

```
<?xml version = "1.0" encoding = "utf-8" ?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- validator2.html
     A document for validator2.js
     Creates text boxes for a name and a phone number
     Note: This document does not work with IE6
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Illustrate form input validation with DOM 2> </title>
<!-- Script to define the event handlers -->
    <script type = "text/javascript"  src = "validator2.js" />
  </head>
  <body>
    <h3> Customer Information </h3>
    <form action = "">
      <p>
        <label>
          <input type = "text"  id = "custName" />
           Name (last name, first name, middle initial)
        </label>
        <br /><br />

        <label>
          <input type = "text"  id = "phone" />
          Phone number (ddd-ddd-dddd)
        </label>
        <br /><br />

        <input type = "reset" />
        <input type = "submit"  id = "submitButton" />
      </p>
    </form>
```

```
<!-- Script for registering event handlers -->
    <script type = "text/javascript"  src = "validator2r.js" />

  </body>
</html>
```

```
// validator2.js
//   An example of input validation using the change and submit
//   events, using the DOM 2 event model
//   Note: This document does not work with IE6

// ************************************************************ //
// The event handler function for the name text box
function chkName(event) {

// Get the target node of the event
  var myName = event.currentTarget;

// Test the format of the input name
//  Allow the spaces after the commas to be optional
//  Allow the period after the initial to be optional
  var pos = myName.value.search(/\w+, ?\w+, ?\w.?/);

  if (pos != 0) {
    alert("The name you entered (" + myName.value +
          ") is not in the correct form. \n" +
          "The correct form is: " +
          "last-name, first-name, middle-initial \n" +
          "Please go back and fix your name");
    myName.focus();
    myName.select();
  }
}

// ************************************************************ //
// The event handler function for the phone number text box
function chkPhone(event) {

// Get the target node of the event
  var myPhone = event.currentTarget;

// Test the format of the input phone number
  var pos = myPhone.value.search(/^\d{3}-\d{3}-\d{4}$/);
```

```
  if (pos != 0) {
    alert("The phone number you entered (" + myPhone.value +
          ") is not in the correct form. \n" +
          "The correct form is: ddd-ddd-dddd \n" +
          "Please go back and fix your phone number");
    myPhone.focus();
    myPhone.select();
  }
}
```

```
// validator2r.js
//   The last part of validator2. Registers the
//   event handlers
//   Note: This script does not work with IE6

// Get the DOM addresses of the elements and register
//  the event handlers
      var customerNode = document.getElementById("custName");
      var phoneNode = document.getElementById("phone");
      customerNode.addEventListener("change", chkName, false);
      phoneNode.addEventListener("change", chkPhone, false);
```

Note that the two event models can be mixed in a document. If a DOM 0 feature happens to be more convenient than the corresponding DOM 2 feature, there is no reason it cannot be used. Chapter 6 includes an example of the use of the DOM 2 event model for something that cannot be done with the DOM 0 event model.

5.9 The navigator Object

The `navigator` object indicates which browser is being used to view the XHTML document. The browser's name is stored in the `appName` property of the `navigator` object. The version of the browser is stored in the `appVersion` property of the `navigator` object. These properties allow the script to determine which browser is being used and to use processes appropri-

ate to that browser. The following example illustrates the use of navigator, in this case just to display the browser name and version number.

```
<?xml version = "1.0"  encoding = "utf-8" ?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- navigate.html
     A document for navigate.js
     Calls the event handler on load
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> navigate.html </title>
    <script type = "text/javascript"  src = "navigate.js" >
    </script>
  </head>
  <body onload = "navProperties()">
  </body>
</html>
```

```
// navigate.js
//  An example of using the navigator object

// The event handler function to display the browser name
//  and its version number
function navProperties() {
  alert("The browser is: " + navigator.appName + "\n" +
    "The version number is: " + navigator.appVersion + "\n");
}
```

Figure 5.9 shows the result of displaying navigate.html with FX2. Figure 5.10 shows the result of displaying navigate.html with IE7. Notice that the version number of IE7 is 4. Microsoft intentionally set the version number to 4 because of some compatibility issues with earlier browsers. One would hope that future versions of IE will use the correct version number. Firefox is not any better in this regard. Using FX2, it displays version 5.0.

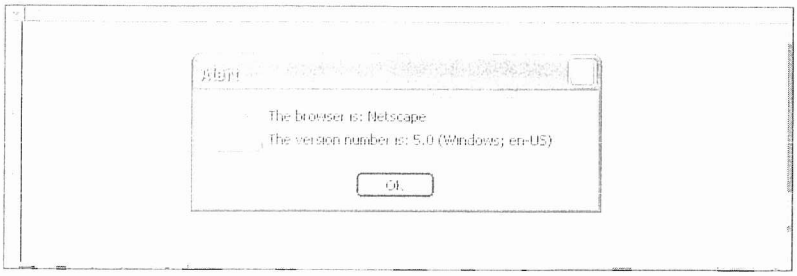

Figure 5.9 The `navigator` properties `appName` and `appVersion` for Firefox 2

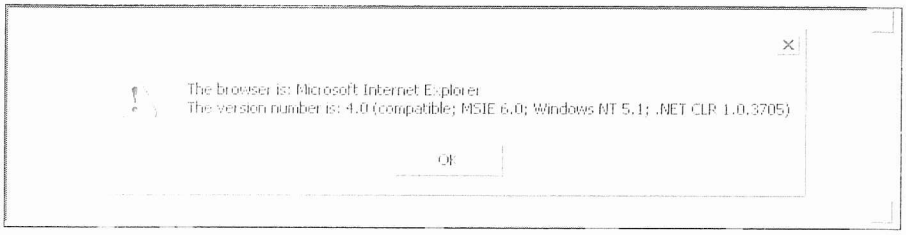

Figure 5.10 The `navigator` properties `appName` and `appVersion` for Internet Explorer 7

5.10 DOM Tree Traversal and Modification

There are many objects, properties, and methods associated with DOM 2 document representations that we have not discussed. One collection of these is defined in the `Element` object and is used to traverse and modify the DOM tree structure of the document being displayed. Every element in the document has an associated `Element` object in the DOM. In this section we briefly describe a few of the most useful of these, without providing any examples of their use. All of the properties and methods mentioned here are supported by both IE7 and FX2.

5.10.1 DOM Tree Traversal

The `parentNode` property has the DOM address of the parent node of the node through which it is referenced. The `previousSibling` property has the DOM address of the previous sibling node of the node through which it is referenced. The `nextSibling` has the DOM address of the previous sibling node of the node through which it is referenced. The `firstChild` and `lastChild` properties have the DOM addresses of the first and last child nodes of the node through which they are referenced. The `nodeType` property has the type of the node through which it is referenced.

5.10.2 DOM Tree Modification

The following methods allow JavaScript code to modify an existing DOM tree structure. The `insertBefore(newChild, refChild)` places the `newChild` node before the `refChild` node. The `replaceChild(newChild, oldChild)` method replaces the `oldChild` with the `newChild` node. The `removeChild(oldChild)` method removes the specified node from the DOM structure. The `appendChild(newChild)` method adds the given node to the end of the list of siblings of the node through which it is called.

Summary

The highest levels of the execution environment of client-side JavaScript are represented with the `Window` and `Document` objects. The `Document` object includes a `forms` array property, which includes references to all forms in the document. Each element of the `forms` array has an `elements` array, which includes references to all elements in the form.

The DOM is an abstract interface whose purpose is to provide a language-independent way to access the elements of an XHTML document. Also included are the means to navigate around the structure in which the XHTML elements appear. XHTML tags are represented in JavaScript as objects; tag attributes are represented as properties.

There are three different ways to access XHTML elements in JavaScript: through the `forms` and `elements` arrays, through the names of the element and its enclosing elements, and through the `getElementById` method.

Events are simply notifications that something specific has happened that may require some special processing. Event-handling code provides that special processing. There are two distinct event models currently in use. The first is the model implemented by all browsers that support JavaScript, which we refer to as the DOM 0 model. The second is the more elaborate and powerful model defined in DOM 2.

With the DOM 0 model, there are two ways to register an event handler. First, an attribute of the tag that defines the XHTML element can be assigned the handler code. Second, the property associated with the event of the object that represents the XHTML element can be assigned the name of a function that implements the handler. The `write` method of `document` should not be used in event handlers.

With the DOM 0 model, each event has an associated tag attribute. A particular attribute may appear in several different tags. Each of these appearances is identified as a different event occurrence. The `load` and `unload` events are often used with the `<body>` tag to perform some operation when a document has been loaded or unloaded, respectively. The `click` event is used for all of the different XHTML buttons, as well as the link of an anchor tag. Form input can be conveniently checked using the `change` event. The `submit` event can also be used to check form data just before the form is submitted.

The DOM 2 event model defines three phases of event processing: capturing, target node, and bubbling. During the capturing phase, the event object travels from the document root to the target node, where the event was created. During the bubbling phase, the event travels back up the document tree to the root, triggering any handlers registered on nodes that are encountered. Event handlers can be set to allow them to be triggered during the capturing phase. Event-handler registration is done with the `addEventListener` method, which sets whether capturing-phase triggering will take place. Events can be unregistered with the `removeEventListener` method. The `currentTarget` property of `Event` has the object through which the handler was called. The `target` property has the target node object. The `mouseEvent` object has two properties, `clientX` and `clientY`, which have the coordinates of the position of the mouse cursor in the browser display window when a mouse event occurs.

The `navigator` object has information about which browser is being used, as well as its version number and other related information.

There are many objects, methods, and properties defined in DOM 2 that are used to traverse and modify the DOM tree structure of a document.

Review Questions

5.1 Global variables in JavaScript are properties of what object?

5.2 How are XHTML elements and attributes represented in the JavaScript binding to DOM?

5.3 What is an event?

5.4 What is an event handler?

5.5 What is the origin of the DOM 0 event model?

5.6 What are the two ways in which an event handler can be associated with an event generated by a specific XHTML element in the DOM 0 event model?

5.7 Why should `document.write` not be used in an event handler?

5.8 In what ways can an XHTML element acquire focus?

5.9 Describe the approach to addressing XHTML elements using `forms` and `elements`.

5.10 Describe the approach to addressing XHTML elements using `name` attributes.

5.11 Describe the approach to addressing XHTML elements using `getElementById`.

5.12 What is the disadvantage of assigning event handlers to event properties?

5.13 What are the advantages of assigning event handlers to event properties?

5.14 Why is it good to use JavaScript to check the validity of form inputs before the form data is sent to the server?

5.15 What three things should be done when a form input element is found to have incorrectly formatted data?

5.16 What exactly does the `focus` function do?

5.17 What exactly does the `select` function do?

5.18 What happens when an event handler for the `onsubmit` event returns `false`?

5.19 What event is used to trigger an event handler that checks the validity of input for a text button in a form?

5.20 What event propagation takes place in the DOM 0 event model?

5.21 Explain the three phases of event processing in the DOM 2 event model.

5.22 Give two examples of default actions of events.

5.23 Explain the first two parameters of the `addEventListener` method.

5.24 How is an event handler registered so that it will be called during the capturing phase?

5.25 How can an event handler be unregistered?

5.26 What exactly do the `clientX` and `clientY` properties store?

5.27 What purpose does the `navigator` object have?

Exercises

5.1 Modify the `radio_click.html` example to have five buttons, labeled *red*, *blue*, *green*, *yellow*, and *orange*. The event handlers for these buttons must produce messages stating the chosen favorite color. The event handler must be implemented as a function, whose name must be assigned to the `onclick` attribute of the radio button elements. The chosen color must be sent to the event handler as a parameter.

5.2 Rewrite the document for Exercise 5.1 to assign the event handler to the event property of the button element. This requires the chosen color to be obtained from the `value` property of the button element rather than through the parameter.

5.3 Develop, test, and validate an XHTML document that has checkboxes for apple (59 cents each), orange (49 cents each), and banana (39 cents each), along with a *Submit* button. Each of the checkboxes should have its own `onclick` event handler. These handlers must add the cost of their fruit to a total cost. An event handler for the *Submit* button must

produce an `alert` window with the message *Your total cost is $xxxx*, where *xxx* is the total cost of the chosen fruit, including 5 percent sales tax. This handler must return `false` (to avoid actual submission of the form data).

5.4 Develop, test, and validate an XHTML document that is similar to that of Exercise 5.3. In this case, use text boxes rather than checkboxes. These text boxes take a number, which is the purchased number of the particular fruit. The rest of the document should behave exactly like that of Exercise 5.3.

5.5 Add reality checks to the text boxes of the document in Exercise 5.4. The checks on the text box inputs should ensure that the input values are numbers in the range of 0 to 99.

5.6 Range checks for element inputs can be represented as new properties of the object that represents the element. Modify the document in Exercise 5.5 to add a `max` property value of 99 and a `min` property value of 0. Your event handler must use the properties for the range checks on values input through the text boxes.

5.7 Develop, test, and validate an XHTML document that collects the following information from the user: last name, first name, middle initial, age (restricted to be greater than 17), and weight (restricted to the range of 80–300). You must have event handlers for the form elements that collect this information that check the input data for correctness. Messages in `alert` windows must be produced when errors are detected.

5.8 Revise the document of Exercise 5.1 to use the DOM 2 event model.

5.9 Revise the document of Exercise 5.3 to use the DOM 2 event model.

CHAPTER 6

Dynamic Documents with JavaScript

6.1 Introduction
6.2 Positioning Elements
6.3 Moving Elements
6.4 Element Visibility
6.5 Changing Colors and Fonts
6.6 Dynamic Content
6.7 Stacking Elements
6.8 Locating the Mouse Cursor
6.9 Reacting to a Mouse Click
6.10 Slow Movement of Elements
6.11 Dragging and Dropping Elements
Summary • Review Questions • Exercises

Informally, a dynamic XHTML document is one that in some way can be changed while it is being displayed by a browser. The most common client-side approach to providing dynamic documents is with JavaScript. Changes to documents can occur when explicitly requested by user interactions, or at regular timed intervals, or when browser events occur.

XHTML elements can be initially positioned at any given location on the browser display. If they're positioned in a specific way, elements can be dynamically moved to new positions on the display. Elements can be made to disappear and reappear. The colors of the background and the foreground (the elements) of a document can be changed. The font, font size, and font style of displayed

text can be changed. Even the content of an element can be changed. Overlapping elements in a document can be positioned in a specific top-to-bottom stacking order, and their stacking order can be dynamically changed. The position of the mouse cursor on the browser display can be determined when the mouse is clicked. Elements can be made to move slowly around the display screen. Finally, elements can be defined to allow the user to drag and drop them anywhere in the display window. This chapter discusses the JavaScript code that can create all of these effects.

6.1 Introduction

Dynamic XHTML is not a new markup language. It is a collection of technologies that allows dynamic changes to documents described with XHTML. Specifically, a *dynamic XHTML document* is one whose tag attributes, tag contents, or element style properties can be changed after the document has been and is still being displayed by a browser. Such changes can be made with an embedded script that accesses the elements of the document as objects in the associated Document Object Model (DOM) structure.

Support for dynamic XHTML is not uniform across the various browsers. As in Chapter 5, "JavaScript and HTML Documents," we restrict our discussion to W3C-standard approaches rather than use features defined by a particular browser vendor. All of the examples in this chapter, except the document in Section 6.11, use the DOM 0 event model and work on both Internet Explorer 7 (IE7) and Firefox 2 (FX2) browsers. The example in Section 6.11 uses the DOM 2 event model because it cannot be designed in a standard way using the DOM 0 event model. Because IE7 does not support the DOM 2 event model, this example does not work with IE7.

This chapter discusses user interactions through XHTML documents using client-side JavaScript. Chapters 9–12 discuss user interactions through XHTML documents using server-side technologies.

6.2 Positioning Elements

Before the browsers that implemented HTML 4.0 appeared, Web site authors had little control over how HTML elements were arranged in documents. In many cases, the elements found in the HTML file were simply placed in the document the way text is placed in a document with a word processor—fill a row, start a new row, fill it, and so forth. HTML tables provide a framework of columns for arranging elements, but they lack flexibility and also take a considerable time to display.[1] This lack of powerful and fast element placement con-

1. Frames provide another way to arrange elements, but they were deprecated in XHTML 1.0 and eliminated in XHTML 1.1.

trol ended when Cascading Style Sheets–Positioning (CSS-P) was released by the W3C in 1997.

CSS-P is completely supported by IE7 and FX2. It provides the means not only to position any element anywhere in the display of a document, but also to move an element to a new position in the display dynamically, using JavaScript to change the positioning style properties of the element. These style properties, which are appropriately named `left` and `top`, dictate the distance from the left and top of some reference point to where the element is to appear. Another style property, `position`, interacts with `left` and `top` to provide a higher level of control of placement and movement of elements. The `position` property has three possible values: `absolute`, `relative`, and `static`.

6.2.1 Absolute Positioning

The `absolute` value is specified for `position` when the element is to be placed at a specific place in the document display without regard to the positions of other elements. For example, if you want a paragraph of text to appear 100 pixels from the left edge of the display window and 200 pixels from the top, you could use the following:

```
<p style = "position: absolute; left: 100px; top: 200px">
    -- text --
</p>
```

One use of absolute positioning is to superimpose special text over a paragraph of ordinary text to create an effect similar to a watermark on paper. A larger italicized font, with space between the letters in a light gray color, could be used for the special text, allowing both the ordinary text and the special text to be legible. The following XHTML document provides an example that implements this approach. In this example, a paragraph of normal text that describes apples is displayed. Superimposed on this paragraph is the somewhat subliminal message "APPLES ARE GOOD FOR YOU."

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- absPos.html
     Illustrates absolute positioning of elements
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Absolute positioning </title>
    <style type = "text/css">
```

```
/* A style for a paragraph of text */
      .regtext {font-family: Times; font-size: 14pt; width: 600px}

/* A style for the text to be absolutely positioned */
     .abstext {position: absolute; top: 25px; left: 50px;
               font-family: Times; font-size: 24pt;
               font-style: italic; letter-spacing: 1em;
               color: rgb(102,102,102); width: 500px}
    </style>
  </head>
  <body>
    <p class = "regtext">
      Apple is the common name for any tree of the genus Malus, of
      the family Rosaceae. Apple trees grow in any of the temperate
      areas of the world. Some apple blossoms are white, but most
      have stripes or tints of rose. Some apple blossoms are bright
      red. Apples have a firm and fleshy structure that grows from
      the blossom. The colors of apples range from green to very
      dark red. The wood of apple trees is fine-grained and hard.
      It is, therefore, good for furniture construction. Apple trees
      have been grown for many centuries. They are propagated by
      grafting because they do not reproduce themselves.
    </p>
    <p class = "abstext">
      APPLES ARE GOOD FOR YOU
    </p>
  </body>
</html>
```

Figure 6.1 shows a display of `absPos.html`.

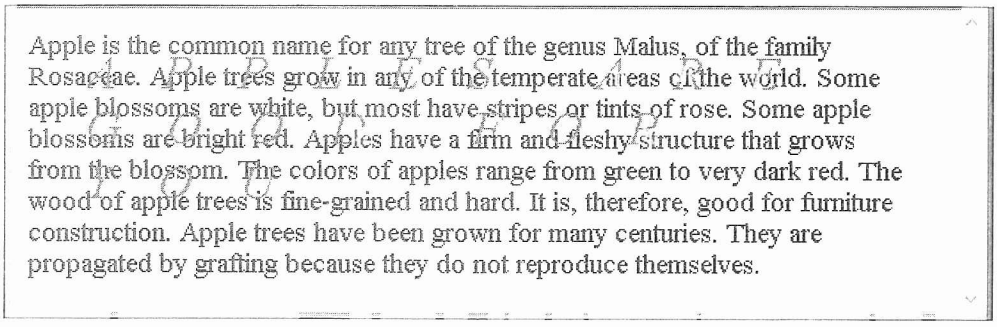

Figure 6.1 Display of `absPos.html`

Notice that a `width` property value is included in the style for both the regular and the special text. This property is used here so we can ensure that the special text is uniformly embedded in the regular text. Without it, the text would extend to the right end of the browser display window. And, of course, the width of the window could vary widely from client to client and even from minute to minute on the same client because the user can resize the browser window at any time.

When an element is absolutely positioned inside another positioned element (one that has the `position` property specified), the `top` and `left` property values are measured from the upper-left corner of the enclosing element (rather than the upper-left corner of the browser window).

To illustrate nested element placement, we modify the document `absPos.html` to place the regular text 100 pixels from the top and 100 pixels from the left. The special text is nested inside the regular text by using `<div>` and `<span>` tags. The modified document, which is named `absPos2.html`, follows:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- absPos2.html
     Illustrates nested absolute positioning of elements
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Nested absolute positioning </title>
    <style type = "text/css">

/* A style for a paragraph of text */
     .regtext {font-family: Times; font-size: 14pt; width: 500px;
               position: absolute; top: 100px; left: 100px;}

/* A style for the text to be absolutely positioned */
     .abstext {position: absolute; top: 25px; left: 50px;
               font-family: Times; font-size: 24pt;
               font-style: italic; letter-spacing: 1em;
               color: rgb(102,102,102); width: 400px;}
    </style>
  </head>
  <body>
    <div class = "regtext">
      Apple is the common name for any tree of the genus Malus, of
      the family Rosaceae. Apple trees grow in any of the temperate
      areas of the world. Some apple blossoms are white, but most
```

```
      have stripes or tints of rose. Some apple blossoms are bright
      red. Apples have a firm and fleshy structure that grows from
      the blossom. The colors of apples range from green to very
      dark red. The wood of apple trees is fine-grained and hard.
     It is, therefore, good for furniture construction. Apple trees
      have been grown for many centuries. They are propagated by
      grafting because they do not reproduce themselves.
      <span class = "abstext">
        APPLES ARE GOOD FOR YOU
      </span>
   </div>
  </body>
</html>
```

Figure 6.2 shows a display of `absPos2.html`.

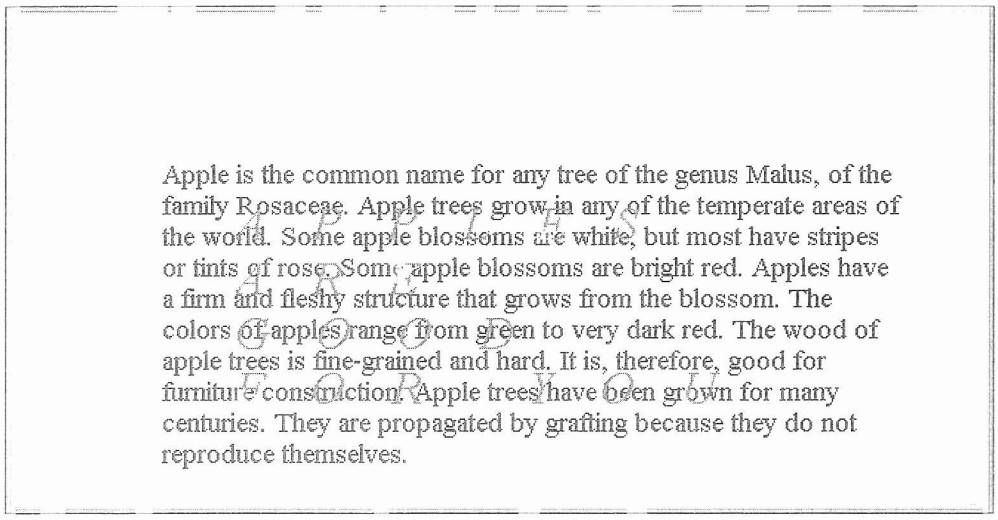

Figure 6.2 Display of `absPos2.html`

6.2.2 Relative Positioning

An element that has the `position` property set to `relative` but does not specify `top` and `left` property values is placed in the document as if the `position` attribute were not set at all. However, such an element can be moved later. If the `top` and `left` properties are given values, they displace the element by the specified amount from the position where it would have been placed (if `top` and `left` had not been set). For example, suppose that two buttons are placed in a document, and the `position` attribute has its default value, which is

static. They would appear next to each other in a row, assuming the current row had sufficient horizontal space for them. If position has been set to relative and the second button had its left property set to 50px, the effect would be to move it 50 pixels farther to the right than it otherwise would have appeared.

In both the case of an absolutely positioned element inside another element and the case of a relatively positioned element, negative values of top and left displace the element upward and to the left, respectively.[2]

Relative positioning can be used for a variety of special effects in element placement. For example, it can be used to create superscripts and subscripts by placing the values to be raised or lowered in tags and displacing them from their regular positions. In the following example, a line of text is set in a normal font style in 24-point size. Embedded in the line is one word that is set in italic, 48-point, red font. Normally, the bottom of the special word would align with the bottom of the rest of the line. In this case, we want the special word to be vertically centered in the line, so its position property is set to relative and its top property is set to 10 pixels, which lowers it by that amount relative to the surrounding text. The XHTML document to specify this, which is named relPos.html, follows:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- relPos.html
     Illustrates relative positioning of elements
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Relative positioning </title>
  </head>
  <body style = "font-family: Times; font-size: 24pt;">
    <p>
      Apples are <span style =
                "position: relative; top: 10px;
                 font-family: Times; font-size: 48pt;
                 font-style: italic; color: red;">
      GOOD </span> for you.
    </p>
  </body>
</html>
```

2. Of course, if the left or top properties are set to negative values for an absolutely positioned element, only part of the element will be visibly displayed.

Figure 6.3 shows a display of `relPos.html`.

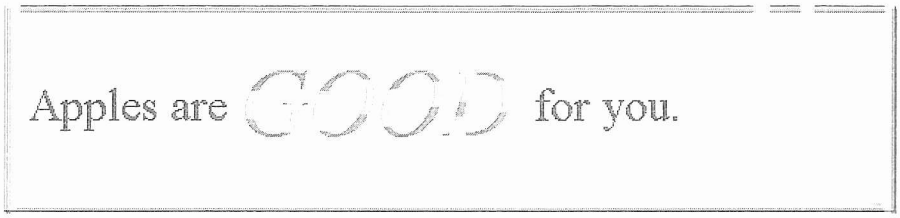

Figure 6.3 Display of `relPos.html`

6.2.3 Static Positioning

The default value for the `position` property is `static`. A statically positioned element is placed in the document as if it had the `position` value of `relative`. The difference is that a statically positioned element cannot have its `top` or `left` properties initially set or changed later. Therefore, a statically placed element cannot be displaced from its normal position and cannot be moved from that position later.

6.3 Moving Elements

As stated previously, an XHTML element whose `position` property is set to either `absolute` or `relative` can be moved. Moving an element is simple: Changing the `top` or `left` property values causes the element to move on the display. If its `position` is set to `absolute`, the element moves to the new values of `top` and `left`; if its `position` is set to `relative`, it moves from its original position by distances given by the new values of `top` and `left`.

In the following example, an image is absolutely positioned in the display. The document includes two text boxes labeled `x coordinate` and `y coordinate`. The user can enter new values for the `left` and `top` properties of the image in these boxes. When the *Move It* button is pressed, the values of the `left` and `top` properties of the image are changed to the given values, and the element is moved to its new position.

A JavaScript function, stored in a separate file, is used to change the values of `left` and `top` in our example. Although it is not necessary in our example, the `id` of the element to be moved is sent to the moving function, just to illustrate that the function could be used on any number of different elements. The values of the two text boxes are also sent to the function as parameters. The actual parameter values are the DOM addresses of the text boxes, with the `value` attribute attached, which provides the complete DOM addresses of the text box values. Notice that we attach `style` to the DOM address of the image to be moved because `top` and `left` are style properties. Because the input `top` and `left` values from the text boxes are just string representations of numbers,

but the `top` and `left` properties must end with some unit abbreviation, the event handler catenates `"px"` to each value before assigning it to the `top` and `left` properties.

```
<?xml version = "1.0" encoding = "utf-8" ?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- mover.html
     Uses mover.js to move an image within a document
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Moving elements </title>
    <script type = "text/javascript"  src = "mover.js" >
    </script>
  </head>
  <body>
    <form action = "">
      <p>
        <label>
          x coordinate:
          <input type = "text"  id = "leftCoord" size = "3" />
        </label>
        <br />
        <label>
          y coordinate:
          <input type = "text"  id = "topCoord" size = "3" />
        </label>
        <br />
        <input type = "button"  value = "Move it"
               onclick =
                 "moveIt('nebula',
                   document.getElementById('topCoord').value,
                   document.getElementById('leftCoord').value)" />
      </p>
    </form>
    <div id = "nebula"  style = "position: absolute;
         top: 115px; left: 0;">
      <img src = "../images/ngc604.jpg"
           alt = "(Picture of a nebula)" />
    </div>
  </body>
</html>
```

```
// mover.js
//   Illustrates moving an element within a document

// The event handler function to move an element
function moveIt(movee, newTop, newLeft) {
  dom = document.getElementById(movee).style;

// Change the top and left properties to perform the move
//  Note the addition of units to the input values
  dom.top = newTop + "px";
  dom.left = newLeft + "px";
}
```

Figures 6.4 and 6.5 show the initial and new positions of an image in `mover.html`.

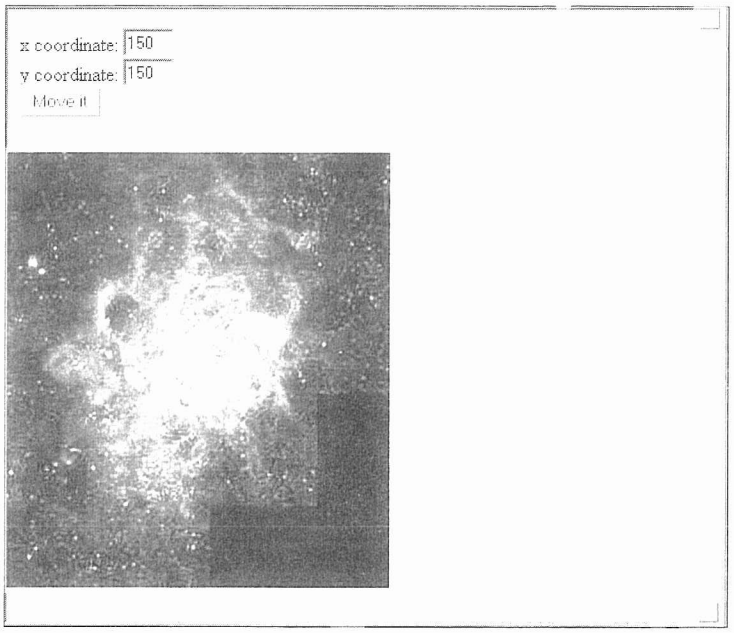

Figure 6.4 Display of `mover.html` (before pressing the *Move It* button)

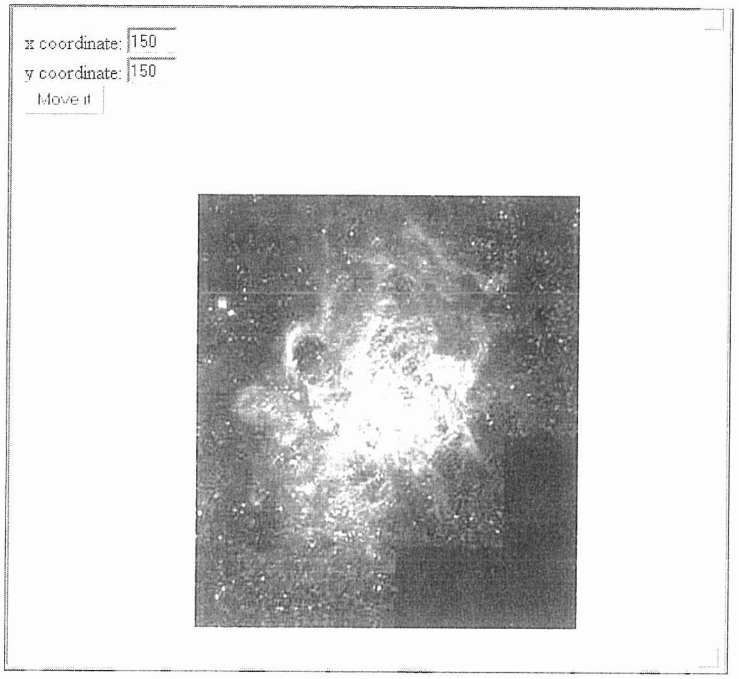

Figure 6.5 Display of `mover.html` (after pressing the *Move It* button)

6.4 Element Visibility

Document elements can be specified to be visible or hidden with the value of their `visibility` property. The two possible values for `visibility` are, quite naturally, `visible` and `hidden`. The appearance or disappearance of an element can be controlled by the user through a widget.

The following example displays an image and allows the user to toggle (with a button) causing the image to appear and not appear in the document display. Once again, the event handler is in a separate file.

```
<?xml version = "1.0"  encoding = "utf-8" ?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- showHide.html
      Uses showHide.js
      Illustrates visibility control of elements
     -->
```

```
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Visibility control </title>
    <script type = "text/javascript"  src = "showHide.js" >
    </script>
  </head>
  <body>
    <form action = "">
      <div id = "saturn"  style = "position: relative;
           visibility: visible;">
        <img src = "../images/saturn.jpg"
             alt = "(Picture of Saturn)" />
      </div>
      <p>
        <br />
        <input type = "button"  value = "Toggle Saturn"
               onclick = "flipImag()" />
      </p>
    </form>
  </body>
</html>
```

```
// showHide.js
//   Illustrates visibility control of elements

// The event handler function to toggle the visibility
//    of the images of Saturn
function flipImag() {
  dom = document.getElementById("saturn").style;

// Flip the visibility adjective to whatever it is not now
 if (dom.visibility == "visible")
   dom.visibility = "hidden";
 else
   dom.visibility = "visible";
}
```

6.5 Changing Colors and Fonts

The background and foreground colors of the document display can be dynamically changed, as can the font properties of the text.

6.5.1 Changing Colors

Dynamic changes to colors are relatively simple. In the following example, the user is presented with two text boxes into which color specifications can be typed—one for the document background color and one for the foreground color. The colors can be specified by any of the three ways that color properties can be given anywhere else. A JavaScript function, which is called (using the `onchange` event) whenever one of the text boxes is changed, makes the change in the document's appropriate color property, `backgroundColor` or `color`. The first of the two parameters to the function specifies whether the new color is for the background or foreground; the second specifies the new color. The new color is the `value` property of the text box that was changed by the user.

In this example, the calls to the handler functions are in the XHTML text box elements. This situation allows a simple way to reference the element's DOM address. The JavaScript `this` variable in this situation is a reference to the object that represents the element in which it is referenced. A reference to such an object is its DOM address. Therefore, in a text element, the value of `this` is the DOM address of the text element. So, in our example, `this.value` is used as an actual parameter to the handler function. Because the call is in an input element, `this.value` is the DOM address of the value of the input element.

```
<?xml version = "1.0"  encoding = "utf-8" ?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- dynColors.html
     Uses dynColors.js
     Illustrates dynamic foreground and background colors
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Dynamic colors </title>
    <script type = "text/javascript"  src = "dynColors.js" >
    </script>
  </head>
  <body>
    <p style = "font-family: Times; font-style: italic;
                font-size: 24pt" >
      This small page illustrates dynamic setting of the
      foreground and background colors for a document
    </p>
    <form action = "">
      <p>
```

```
        <label>
          Background color:
          <input type = "text"  name = "background" size = "10"
                 onchange = "setColor('background', this.value)" />
        </label>
        <br />
        <label>
          Foreground color:
          <input type = "text"  name = "foreground" size = "10"
                 onchange = "setColor('foreground', this.value)" />
        </label>
        <br />
      </p>
    </form>
  </body>
</html>
```

```
// dynColors.js
//   Illustrates dynamic foreground and background colors

// The event handler function to dynamically set the
// color of background or foreground
function setColor(where, newColor) {
  if (where == "background")
    document.body.style.backgroundColor = newColor;
  else
    document.body.style.color = newColor;
}
```

6.5.3 Changing Fonts

Web users are accustomed to having links in documents change color when the cursor is placed over them. Any property of a link can be changed by using the mouse event `mouseover` to trigger JavaScript event handlers. Thus, the font style and font size, as well as the color and background color of a link, can be changed when the cursor is placed over the link. The link can be changed back to its original form when an event handler is triggered with the `mouseout` event. In the following example, the only element is a sentence with an embedded link. The foreground color for the document is the default black. The link is presented in blue. When the mouse cursor is placed over the link, its color changes to red and its font style changes to italic. Notice that the event handlers in this example are embedded in the markup. This is one of those cases where the small amount of JavaScript needed does not justify putting it in a separate file.

```
<?xml version = "1.0"  encoding = "utf-8" ?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- dynLink.html
     Illustrates dynamic font styles and colors for links
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Dynamic fonts for links </title>
    <style type = "text/css">
      .regText {font: Times; font-size: 16pt;}
    </style>
  </head>
  <body>
    <p class = "regText">
      The state of
      <a style = "color: blue;"
         onmouseover = "this.style.color = 'red';
                        this.style.font = 'italic 16pt Times';"
         onmouseout = "this.style.color = 'blue';
                       this.style.font = 'normal 16pt Times';">
         Washington
      </a>
      produces many of our nation's apples.
    </p>
  </body>
</html>
```

Figures 6.6 and 6.7 show browser displays of the `dynLink.html` document with the mouse cursor not over and then over the link.

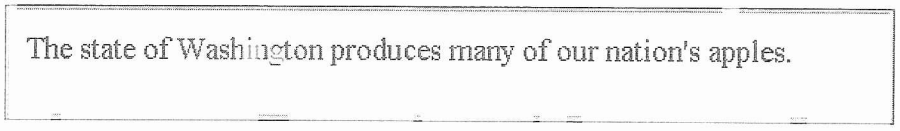

Figure 6.6 Display of `dynLink.html` with the cursor not over the link

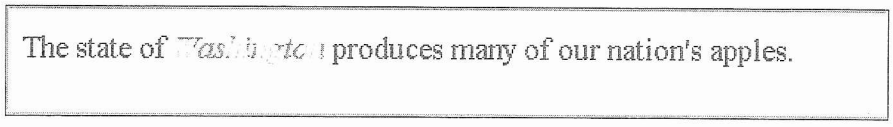

Figure 6.7 Display of `dynLink.html` with the mouse cursor over the link

6.6 Dynamic Content

We have explored the options of dynamically changing the positions of elements, their visibility, and the colors, background colors, and styles of text fonts. This section investigates changing the content of XHTML elements. The content of an element is accessed through the `value` property of its associated JavaScript object. So, changing the content of an element is not essentially different from changing other properties of the element. The following example illustrates changing the content of a collection of text fields:

```
<?xml version = "1.0" encoding = "utf-8" ?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- dynValue.html
     Uses dynValue.js
     Illustrates dynamic values
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Dynamic values </title>
    <script type = "text/javascript"  src = "dynValue.js" >
    </script>
  </head>
  <body>
    <form action = "">
      <p style = "font-weight: bold">
        <span style = "font-style: italic">
          Customer information
        </span>
        <br /><br />
        <label>
          Name:
          <input type = "text"  onmouseover = "messages(0)"
                 onmouseout = "messages(4)" />
        </label>
        <br />
        <label>
          Email:
          <input type = "text"  onmouseover = "messages(1)"
                 onmouseout = "messages(4)" />
        </label>
        <br /> <br />
        <span style = "font-style: italic">
          To create an account, provide the following:
```

```
        </span>
        <br /> <br />
        <label>
          User ID:
          <input type = "text"  onmouseover = "messages(2)"
                 onmouseout = "messages(4)" />
        </label>
        <br />
        <label>
          Password:
          <input type = "password"
                 onmouseover = "messages(3)"
                 onmouseout = "messages(4)" />
        </label>
        <br />
        <textarea id = "adviceBox"  rows = "3"  cols = "50"
                  style = "position: absolute; left: 250px;
                  top: 0px">
This box provides advice on filling out the form
on this page. Put the mouse cursor over any input
field to get advice.
        </textarea>
        <br /><br />
        <input type = "submit"  value = "Submit" />
        <input type = "reset"  value = "Reset" />
      </p>
    </form>
  </body>
</html>
```

```
// dynValue.js
//   Illustrates dynamic values

var helpers = ["Your name must be in the form: \n \
 first name, middle initial., last name",
  "Your email address must have the form: \
 user@domain",
  "Your user ID must have at least six characters",
  "Your password must have at least six \
 characters and it must include one digit",
  "This box provides advice on filling out\
 the form on this page. Put the mouse cursor over any \
 input field to get advice"]
```

```
// ********************************************************
// The event handler function to change the value of the
//  textarea

function messages(adviceNumber) {
  document.getElementById("adviceBox").value =
                                        helpers[adviceNumber];
}
```

Assistance to a user filling out a form can be provided by an associated text area, often called a *help box*. The content of the help box can change, depending on the placement of the mouse cursor. When the cursor is placed over a particular input field, the help box can display advice on how the field is to be filled in. When the cursor is moved away from an input field, the help box content can be changed to simply indicate that assistance is available.

In the previous example, an array of messages that can be displayed in the help box is defined in JavaScript. Note that the backslash characters that terminate some of the lines of the literal array of messages specify that the string literal is continued on the next line. When the mouse cursor is placed over an input field, the `mouseover` event is used to call a function that changes the help box content to the appropriate value (the one associated with the input field). The appropriate value is specified with a parameter sent to the handler function. The `mouseout` event is used to trigger the change of the content of the help box back to the "standard" value.

Figure 6.8 shows a browser display of the page defined by `dynValue.html` when the mouse cursor is over the `User ID` input field.

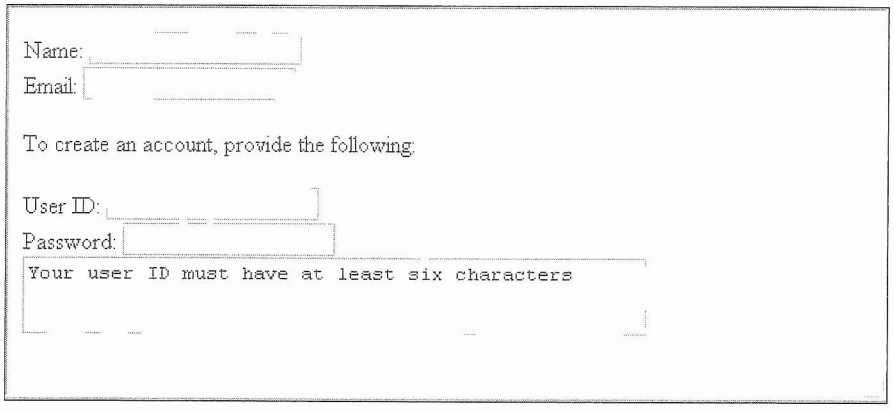

Figure 6.8 Display of `dynValue.html`

6.7 Stacking Elements

The `top` and `left` properties allow the placement of an element anywhere in the two dimensions of the display of a document. Although the display is restricted to two physical dimensions, the effect of a third dimension is possible through the simple concept of stacked elements, such as that used to stack windows in windowing systems. Although multiple elements can occupy the same space in the document, one is considered to be on top and is displayed. The top element hides the parts of the lower elements on which it is superimposed. The placement of elements in this third dimension is controlled by the `z-index` attribute of the element. An element whose `z-index` is greater than that of an element in the same space will be displayed over the other element, effectively hiding the element with the smaller `z-index` value. The JavaScript style property associated with the `z-index` attribute is `zIndex`.

In the following example, three images are placed on the display so that they overlap. In the XHTML description of this, each image tag includes an `onclick` attribute, which is used to trigger the execution of a JavaScript handler function. First the function defines DOM addresses for the last top element and the new top element. Then the function sets the `zIndex` value of the two elements so that the old top element has a value of `0` and the new top element has the value `10`, effectively putting it at the top. The script keeps track of which image is currently on top with the global variable `top`, which is changed every time a new element is moved to the top with the `toTop` function. Note that the `zIndex` value, as with other properties, is a string.

```
<?xml version = "1.0"  encoding = "utf-8" ?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- stacking.html
     Uses stacking.js
     Illustrates dynamic stacking of images
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Dynamic stacking of images </title>
    <script type = "text/javascript"  src = "stacking.js" >
    </script>
    <style type = "text/css">
      .plane1 {position: absolute; top: 0; left: 0;
               z-index: 0;}
      .plane2 {position: absolute; top: 50px; left: 110px;
               z-index: 0;}
```

```
      .plane3 {position: absolute; top: 100px; left: 220px;
               z-index: 0;}
    </style>
  </head>
  <body>
    <p>
      <img class = "plane1"  id = "C172"
           src = "../images/c172.gif"
           alt = "(Picture of a C172)"
           onclick = "toTop('C172')" />
      <img class = "plane2"  id = "cix"
           src = "../images/cix.gif"
           alt = "(Picture of a Citation airplane)"
           onclick = "toTop('cix')" />
      <img class = "plane3"  id = "C182"
           src = "../images/c182.gif"
           alt = "(Picture of a C182)"
           onclick = "toTop('C182')" />
    </p>
  </body>
</html>
```

```
// stacking.js
//   Illustrates dynamic stacking of images
var top = "C172";

// The event handler function to move the given element
//  to the top of the display stack
function toTop(newTop) {

// Set the two dom addresses, one for the old top
//  element and one for the new top element
  domTop = document.getElementById(top).style;
  domNew = document.getElementById(newTop).style;

// Set the zIndex properties of the two elements, and
//  reset top to the new top
  domTop.zIndex = "0";
  domNew.zIndex = "10";
  top = newTop;
}
```

Figures 6.9, 6.10, and 6.11 show the document described by `stacking.html` in three of its possible configurations.

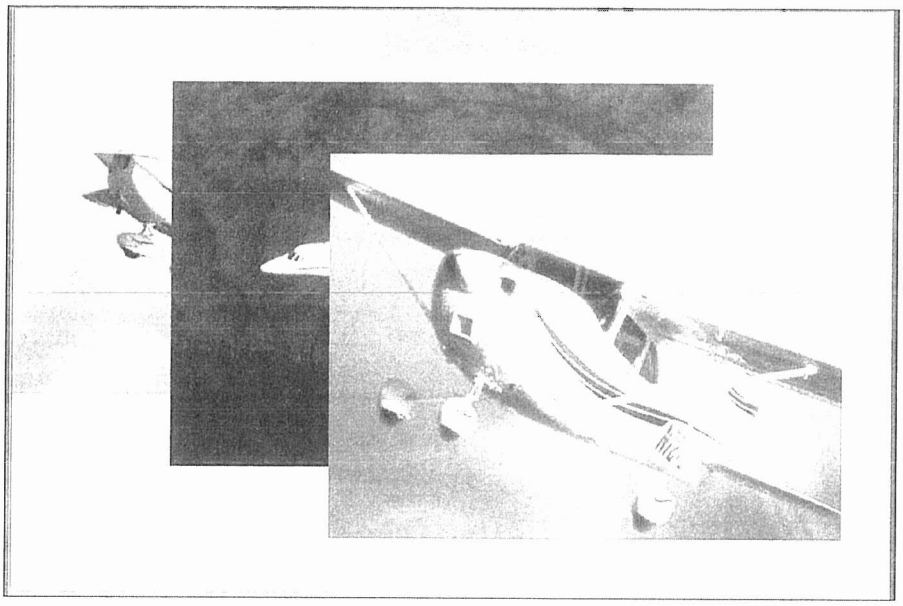

Figure 6.9 The initial display of `stacking.html` (photographs courtesy of Cessna Aircraft Company)

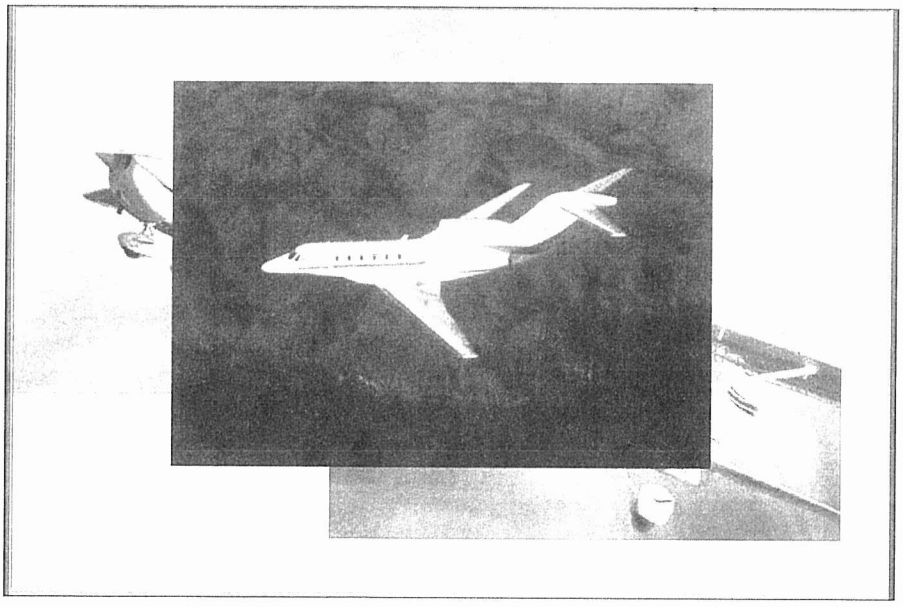

Figure 6.10 The display of `stacking.html` after clicking the second image (photographs courtesy of Cessna Aircraft Company)

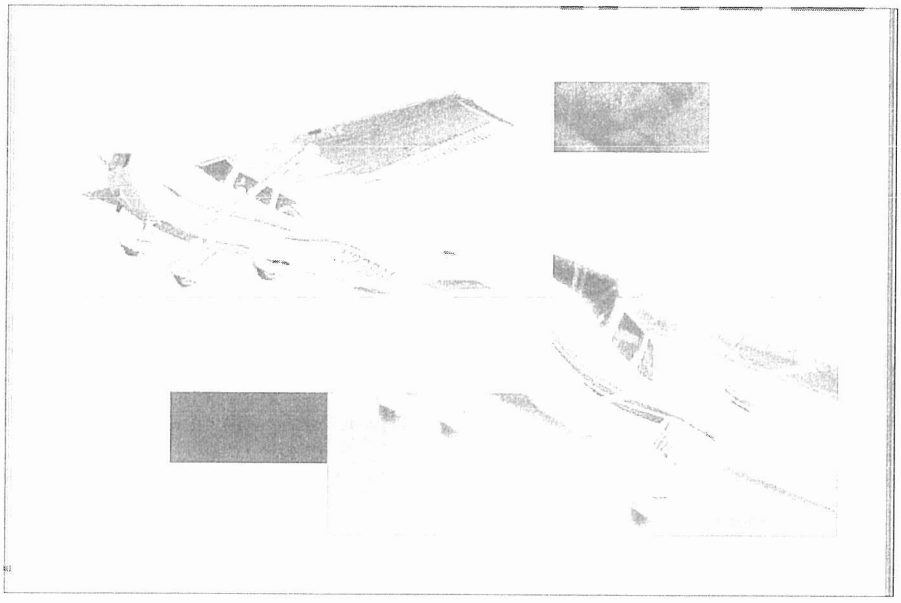

Figure 6.11 The display of `stacking.html` after clicking the bottom image (photographs courtesy of Cessna Aircraft Company)

6.8 Locating the Mouse Cursor

Recall from Chapter 5 that every event that occurs in an XHTML document creates an event object. This object includes some information about the event. A mouse-click event is an implementation of the `MouseEvent` interface, which defines two pairs of properties that provide geometric coordinates of the position of the element in the display that created the event. One of these pairs, `clientX` and `clientY`, gives the coordinates of the element relative to the upper-left corner of the browser display window, in pixels. The other pair, `screenX` and `screenY`, also gives coordinates of the element but relative to the client computer's screen. Obviously, the former pair is usually more useful than the latter.

In the following example, `where.html`, two pairs of text boxes are used to display these four properties every time the mouse button is clicked. The handler is triggered by the `onclick` attribute of the body element. An image is displayed just below the display of the coordinates, but only to make the screen more interesting.

The call to the handler in this example sends `event`, which is a reference to the event just created in the element, as a parameter. This is a bit of magic, because the event object is implicitly created. In the handler, the formal parameter is used to access the coordinate properties. Note that the handling of the event object is not implemented the same way in the popular browsers. The

Firefox browsers send it as a parameter to event handlers, whereas Microsoft browsers make it available as a global property. The code in `where.html` works for both of these approaches by sending it in the call to the handler. It is available in the call with Microsoft browsers because it is visible there as a global variable. Of course, for a Microsoft browser, it need not be sent at all.

```
<?xml version = "1.0"  encoding = "utf-8" ?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- where.html
     Uses where.js
     Illustrates x and y coordinates of the mouse cursor
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Where is the cursor? </title>
    <script type = "text/javascript"  src = "where.js" >
    </script>
  </head>
  <body onclick = "findIt(event)">
    <form action = "">
      <p>
        Within the client area: <br />
        x:
        <input type = "text"  id = "xcoor1"  size = "4" />
        y:
        <input type = "text"  id = "ycoor1"  size = "4" />
        <br /><br />
        Relative to the origin of the screen coordinate system:
        <br />
        x:
        <input type = "text"  id = "xcoor2"  size = "4" />
        y:
        <input type = "text"  id = "ycoor2"  size = "4" />
      </p>
    </form>
    <p>
      <img src = "../images/c172.gif"  alt = "(Picture of C172)" />
    </p>
  </body>
</html>
```

```
// where.js
//   Show the coordinates of the mouse cursor position
//   in an image and anywhere on the screen when the mouse
//   is clicked

// The event handler function to get and display the
//  coordinates of the cursor, both in an element and
//  on the screen
function findIt(evt) {
  document.getElementById("xcoor1").value = evt.clientX;
  document.getElementById("ycoor1").value = evt.clientY;
  document.getElementById("xcoor2").value = evt.screenX;
  document.getElementById("ycoor2").value = evt.screenY;
}
```

Figure 6.12 shows a browser display of `where.html`.

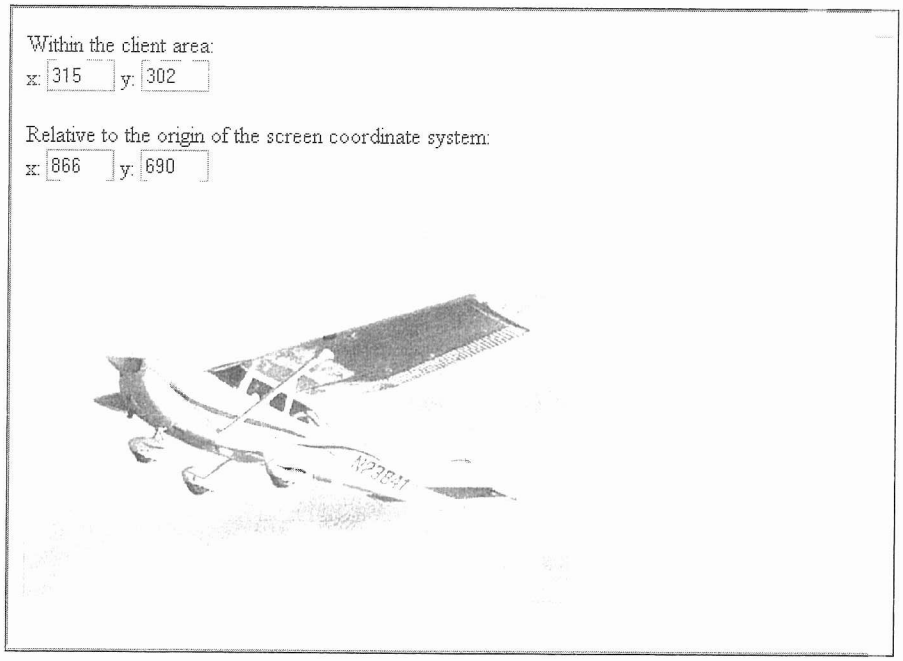

Figure 6.12 Display of `where.html` (the cursor was in the tail section of the plane)

One interesting note about the cursor-finding example is that with IE7, the mouse clicks are ignored if the mouse cursor is below the last element on the display. The FX2 browser always responds the same way regardless of where the cursor is on the display.

6.9 Reacting to a Mouse Click

The following is another example related to reacting to mouse clicks. In this case, the `mousedown` and `mouseup` events are used to show and hide the message "Please don't click here!" on the display under the mouse cursor whenever the mouse button is clicked, regardless of where the cursor is at the time. The offsets (`-130` for `left` and `-25` for `top`) modify the actual cursor position so that the message is approximately centered over the cursor position.

```
<?xml version = "1.0"  encoding = "utf-8" ?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- anywhere.html
     Uses anywhere.js
     Display a message when the mouse button is pressed,
     no matter where it is on the screen
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Sense events anywhere </title>
    <script type = "text/javascript"  src = "anywhere.js" >
    </script>
  </head>
  <body onmousedown = "displayIt(event);"
        onmouseup = "hideIt();">
    <p>
      <span id= "message"
            style = "color: red; visibility: hidden;
                     position: relative;
                     font-size: 20pt; font-style: italic;
                     font-weight: bold;">
         Please don't click here!
      </span>
      <br /><br /><br /><br /><br /><br /><br />
      <br /><br /><br /><br /><br /><br /><br />
    </p>
  </body>
</html>
```

```
// anywhere.js
//   Display a message when the mouse button is pressed,
//   no matter where it is on the screen

// The event handler function to display the message
function displayIt(evt) {
  var dom = document.getElementById("message");
  dom.style.left = (evt.clientX - 130) + "px";
  dom.style.top = (evt.clientY - 25) + "px";
  dom.style.visibility = "visible";
}

// ************************************************
// The event handler function to hide the message
function hideIt() {
  document.getElementById("message").style.visibility =
      "hidden";
}
```

6.10 Slow Movement of Elements

So far, we have considered only element movements that happen instantly. These movements are controlled by changing the `top` and `left` properties of the element to be moved. The only way to move an element slowly is to move it by small amounts many times, with the moves separated by small amounts of time. JavaScript has two `Window` methods that are capable of this: `setTimeout` and `setInterval`.

The `setTimeout` method takes two parameters: a string of JavaScript code to be executed and a number of milliseconds of delay before executing the given code. For example, consider the following call:

```
setTimeout("mover()", 20);
```

This causes a 20-millisecond delay, after which the function `mover` is called.

The `setInterval` method has two forms. One form takes two parameters, exactly as does `setTimeout`. It executes the given code repeatedly, using the second parameter as the interval in milliseconds between executions. The second form of `setInterval` takes a variable number of parameters. The first parameter is the name of a function to be called, the second is the interval in milliseconds between the calls to the function, and the remaining parameters are used as actual parameters to the function being called.

The example presented here, `moveText.html`, moves a string of text from one position (100, 100) to a new position (300, 300). The move is accomplished by using `setTimeout` to call a mover function every millisecond until the final position (300, 300) is reached. The initial position of the text is set in the span

element that specifies the text. The `onload` attribute of the body element is used to call a function, `initText`, to initialize the *x* and *y* coordinates for the initial position to the `left` and `top` properties of the element and call the mover function.

The mover function, named `moveText`, takes the current coordinates of the text as parameters, moves them 1 pixel toward the final position, and then calls itself with the new coordinates using `setTimeout`. The recomputation of the coordinates is complicated by the fact that we want the code to work regardless of the direction of the move.

One consideration with this script is that the coordinate properties are stored as strings with units attached. For example, if the initial position of an element is (100, 100), its `left` and `top` property values both have the string value `"100px"`. To change the properties arithmetically, we must have them as numbers. Therefore, the property values are converted to numbers in the `initText` function by stripping the nondigit unit parts. Then, before the `left` and `top` properties are set to the new coordinates, the units (in this case, `"px"`) are catenated back on to the coordinates.

It is interesting that in this example, placing the event handler in a separate file avoids a problem that would occur if the JavaScript were embedded in the markup. The problem is the use of XHTML comments to hide JavaScript and having possible parts of XHTML comments embedded in the JavaScript. For example, if the JavaScript statement `x--;` is embedded in an XHTML comment, the validator complains that the `--` in the statement is an invalid comment declaration. In the JavaScript code of the following example, the statement `x--;` is used to move the x-coordinate of the text being moved.

In the code file, `moveText.js`, note the complexity of the call to the `moveText` function in the call to `setTimeout`. This is required because the call to `moveText` must be built from static strings with the values of the variables `x` and `y` catenated in.

The JavaScript script for `moveText.html` is as follows:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- moveText.html
     Uses moveTextfuns.js
     Illustrates a moving text element
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Moving text </title>
     <script type = "text/javascript"
             src = "moveTextfuns.js">
     </script>
```

```
  </head>
<!-- Call the initializing function on load, giving the
      destination coordinates for the text to be moved
      -->
  <body onload = "initText()">

<!-- The text to be moved, including its initial position -->
    <p>
      <span id = 'theText' style =
                "position: absolute; left: 100px; top: 100px;
                 font: bold 20pt 'Times Roman';
                 color: blue;"> Jump in the lake!
      </span>
    </p>
  </body>
</html>

//*****************************************************************
// This is moveTextfuns.js - used with moveText.html
   var dom, x, y, finalx = 300, finaly = 300;

// ********************************************** //
// A function to initialize the x and y coordinates
//  of the current position of the text to be moved,
//  and then call the mover function
   function initText() {
      dom = document.getElementById('theText').style;

   /* Get the current position of the text */
      var x = dom.left;
      var y = dom.top;

   /* Convert the string values of left and top to
      numbers by stripping off the units */
      x = x.match(/\d+/);
      y = y.match(/\d+/);

   /* Call the function that moves it */
      moveText(x, y);
   } /*** end of function initText */

// ********************************************** //
// A function to move the text from its original
//  position to (finalx, finaly)
   function moveText(x, y) {
```

```
/* If the x coordinates are not equal, move
    x toward finalx */
   if (x != finalx)
      if (x > finalx) x--;
      else if (x < finalx) x++;

/* If the y coordinates are not equal, move
    y toward finaly */
   if (y != finaly)
      if (y > finaly) y--;
      else if (y < finaly) y++;

/* As long as the text is not at the destination,
    call the mover with the current position */
   if ((x != finalx) || (y != finaly)) {

/* Put the units back on the coordinates before
   assigning them to the properties to cause the
   move */
      dom.left = x + "px";
      dom.top = y + "px";

/* Recursive call, after a 1-millisecond delay */
      setTimeout("moveText(" + x + "," + y + ")", 1);
   }

 } /*** end of function moveText */
```

6.11 Dragging and Dropping Elements

One of the more powerful effects of event handling is allowing the user to drag and drop elements around the display screen. The `mouseup`, `mousedown`, and `mousemove` events can be used to implement this. Changing the `top` and `left` properties of an element, as we saw earlier in this chapter, causes the element to move. To illustrate drag and drop, we develop an example that creates a magnetic poetry system, showing two static lines of a poem and allowing the user to create the last two lines from a collection of movable words.

This example uses a mixture of the DOM 0 and DOM 2 event models. The DOM 0 model is used for the call to the handler for the `mousedown` event. The rest of the process is designed with the DOM 2 model. The `mousedown` event handler, `grabber`, takes the `Event` object as its parameter. It gets the element to be moved from the `currentTarget` property of the `Event` object and puts it in a global variable so it is available to the other handlers. Then it determines the coordinates of the current position of the element to be moved and com-

putes the difference between them and the coordinates of the position of the mouse cursor. These two differences, which are used by the handler for `mousemove` to actually move the element, are also placed in global variables. The `grabber` handler also registers the event handlers for `mousemove` and `mouseup`. These two handlers are named `mover` and `dropper`, respectively. The `dropper` handler disconnects mouse movements from the element-moving process by unregistering the handlers `mover` and `dropper`. The following is the document we have just described:

```
<?xml version = "1.0"  encoding = "utf-8" ?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- dragNDrop.html
     An example to illustrate the DOM 2 Event model
     Allows the user to drag and drop words to complete
     a short poem
     Does not work with IE7
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Drag and drop </title>
    <script type = "text/javascript"  src = "dragNdrop.js" >
    </script>
  </head>
  <body style = "font-size: 20;">
    <p>
      Roses are red <br />
      Violets are blue <br />

      <span style = "position: absolute; top: 200px; left: 0px;
                     background-color: lightgrey;"
            onmousedown = "grabber(event);"> candy </span>
      <span style = "position: absolute; top: 200px; left: 75px;
                     background-color: lightgrey;"
            onmousedown = "grabber(event);"> cats </span>
      <span style = "position: absolute; top: 200px; left: 150px;
                     background-color: lightgrey;"
            onmousedown = "grabber(event);"> cows </span>
      <span style = "position: absolute; top: 200px; left: 225px;
                     background-color: lightgrey;"
            onmousedown = "grabber(event);"> glue </span>
      <span style = "position: absolute; top: 200px; left: 300px;
                     background-color: lightgrey;"
            onmousedown = "grabber(event);"> is </span>
```

```
      <span style = "position: absolute; top: 200px; left: 375px;
                     background-color: lightgrey;"
            onmousedown = "grabber(event);"> is </span>
      <span style = "position: absolute; top: 200px; left: 450px;
                     background-color: lightgrey;"
            onmousedown = "grabber(event);"> meow </span>
      <span style = "position: absolute; top: 250px; left: 0px;
                     background-color: lightgrey;"
            onmousedown = "grabber(event);"> mine </span>
      <span style = "position: absolute; top: 250px; left: 75px;
                     background-color: lightgrey;"
            onmousedown = "grabber(event);"> moo </span>
      <span style = "position: absolute; top: 250px; left: 150px;
                     background-color: lightgrey;"
            onmousedown = "grabber(event);"> new </span>
      <span style = "position: absolute; top: 250px; left: 225px;
                     background-color: lightgrey;"
            onmousedown = "grabber(event);"> old </span>
      <span style = "position: absolute; top: 250px; left: 300px;
                     background-color: lightgrey;"
            onmousedown = "grabber(event);"> say </span>
      <span style = "position: absolute; top: 250px; left: 375px;
                     background-color: lightgrey;"
            onmousedown = "grabber(event);"> say </span>
      <span style = "position: absolute; top: 250px; left: 450px;
                     background-color: lightgrey;"
            onmousedown = "grabber(event);"> so </span>
      <span style = "position: absolute; top: 300px; left: 0px;
                     background-color: lightgrey;"
            onmousedown = "grabber(event);"> sticky </span>
      <span style = "position: absolute; top: 300px; left: 75px;
                     background-color: lightgrey;"
            onmousedown = "grabber(event);"> sweet </span>
      <span style = "position: absolute; top: 300px; left: 150px;
                     background-color: lightgrey;"
            onmousedown = "grabber(event);"> syrup </span>
      <span style = "position: absolute; top: 300px; left: 225px;
                     background-color: lightgrey;"
            onmousedown = "grabber(event);"> too </span>
      <span style = "position: absolute; top: 300px; left: 300px;
                     background-color: lightgrey;"
            onmousedown = "grabber(event);"> yours </span>
    </p>
  </body>
</html>
```

```
// dragNDrop.js
//   An example to illustrate the DOM 2 Event model
//   Allows the user to drag and drop words to complete
//   a short poem
//   Does not work with IE7

// Define variables for the values computed by
//  the grabber event handler but needed by mover
//  event handler
      var diffX, diffY, theElement;

// ******************************************************
// The event handler function for grabbing the word
function grabber(event) {

// Set the global variable for the element to be moved
  theElement = event.currentTarget;

// Determine the position of the word to be grabbed,
//  first removing the units from left and top
  var posX = parseInt(theElement.style.left);
  var posY = parseInt(theElement.style.top);

// Compute the difference between where it is and
//  where the mouse click occurred
  diffX = event.clientX - posX;
  diffY = event.clientY - posY;

// Now register the event handlers for moving and
//  dropping the word
  document.addEventListener("mousemove", mover, true);
  document.addEventListener("mouseup", dropper, true);

// Stop propagation of the event and stop any default
//  browser action
  event.stopPropagation();
  event.preventDefault();

}  //** end of grabber

// ******************************************************
// The event handler function for moving the word
function mover(event) {

// Compute the new position, add the units, and move the word
```

```
  theElement.style.left = (event.clientX - diffX) + "px";
  theElement.style.top = (event.clientY - diffY) + "px";

// Prevent propagation of the event
  event.stopPropagation();
}  //** end of mover

// ********************************************************
// The event handler function for dropping the word
function dropper(event) {

// Unregister the event handlers for mouseup and mousemove
  document.removeEventListener("mouseup", dropper, true);
  document.removeEventListener("mousemove", mover, true);

// Prevent propagation of the event
  event.stopPropagation();
}  //** end of dropper
```

Figure 6.13 shows a browser display of `dragNDrop.html`.

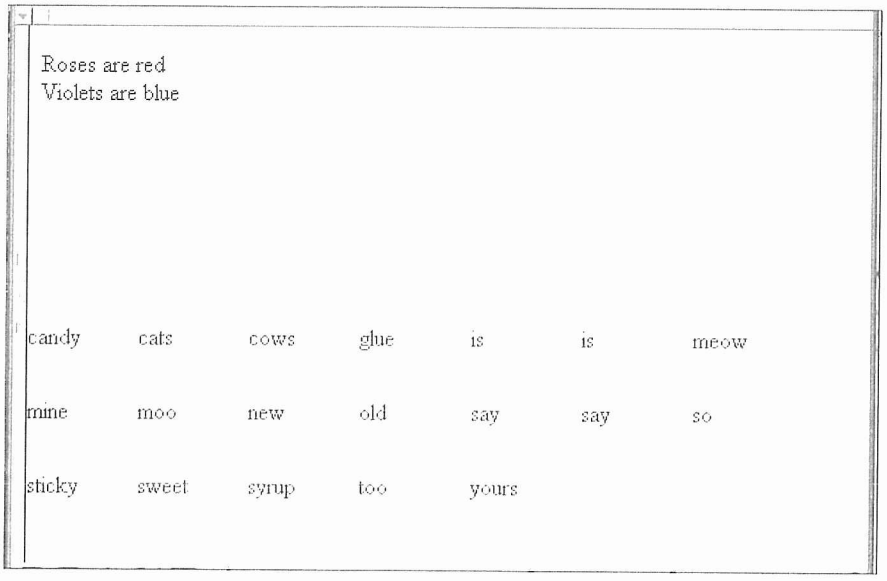

Figure 6.13 Display of `dragNDrop.html`

Note that the drag-and-drop process can be written with the DOM 0 event model. However, it can only be made portable by having the script detect which browser is being used and using different code for the different browsers. We

have chosen to write it with the DOM 2 event model rather than deal with that untidy situation. Before long, we hope all browsers will implement the DOM 2 model.

Summary

The CSS-P standard enables us initially to place XHTML elements wherever we want in a document and then move them later. Elements can be positioned at any given location in the display of a document if their `position` property is set to `absolute` or `relative`. Absolute positioning is used to place an element at a position in the display of the document relative to the upper-left corner of the display, using the `left` and `top` properties of the element. Relative positioning is used to place an element at a specified offset from the `top` and `left` coordinates of where it would have gone with the default static positioning. Relative positioning also allows an element to be moved later. Static positioning, which is the default, disallows both specific initial placement and dynamic moving of the element. An XHTML element can be made to disappear and reappear by changing its `visibility` property.

The color of the background of a document is stored in its `backgroundColor` property; the color of an element is stored in its `color` property. Both of these can be dynamically changed. The font, font size, and font style of text also can be changed.

The content of an element can be changed by changing its `value` property. An element in a document can be set to appear to be in front of other elements, and this top-to-bottom stacking order can be dynamically changed. The coordinates of the mouse cursor can be found every time a mouse button is pressed, using properties of the event object. An element can be animated, at least in a crude way, by changing its `top` and `left` properties repeatedly by small amounts. Such an operation can be controlled by the `Window` method `setTimeout`. Event handlers for the mouse events can be written to allow elements to be dragged and dropped anywhere on the display screen by the user.

Review Questions

6.1 Define a dynamic XHTML document.

6.2 If you know the `id` of an XHTML element, how can you get the DOM address of that element in JavaScript?

6.3 If you have a variable that has the `id` of an XHTML element, how can you get the DOM address of that element in JavaScript?

6.4 In what additional way can you obtain the DOM addresses of radio buttons and checkboxes?

6.5 What is CSS-P?

6.6 Describe all of the differences between the three possible values of the `position` property.

6.7 What are the standard values for the `visibility` property?

6.8 What properties control the foreground and background colors of a document?

6.9 What events can be used to change a font when the mouse cursor is moved over and away from an element?

6.10 What property has the content of an element?

6.11 What JavaScript variable is associated with the `z-index` property?

6.12 To move an element to the top of the display, do you set its `z-index` property to a large number or a small number?

6.13 What exactly is stored in the `clientX` and `clientY` properties after a mouse click?

6.14 What exactly is stored in the `screenX` and `screenY` properties after a mouse click?

6.15 Describe the parameters and actions of the `setTimeout` function.

Exercises

Write, test, validate, and debug (if necessary) the following documents:

6.1 The document must have a paragraph of at least 10 lines of text that describe you. This paragraph must be centered on the page and have space for 20 characters per line only. A light gray image of yourself must be superimposed over the center of the text as a nested element.

6.2 Modify the document described in Exercise 6.1 to add four buttons. These buttons must be labeled *Northwest*, *Northeast*, *Southwest*, and *Southeast*. When they're pressed, the buttons must move your image to the specified corner of the text. Initially, your image must appear in the northwest (upper-left) corner of the text.

6.3 Modify the document described in Exercise 6.2 to make the buttons toggle their respective copies of your image on and off so that, at any time, the document may include none, one, two, three, or four copies of your image. The initial document should have no images shown.

6.4 The document must have a paragraph of text that describes your home. Choose at least three different phrases (three to six words) of this paragraph and make them change font, font style, color, and font size when the mouse cursor is placed over them. Each of the different phrases must change to different fonts, font styles, colors, and font sizes.

6.5 The document must display an image and three buttons. The buttons should be labeled simply *1*, *2*, and *3*. When pressed, each button should change the content of the image to that of a different image.

6.6 The document must contain four short paragraphs of text, stacked on top of each other, with only enough of each showing so that the mouse cursor can always be placed over some part of them. When the cursor is placed over the exposed part of any paragraph, it should rise to the top to become completely visible.

6.7 Modify the document of Exercise 6.6 so that when a paragraph is moved from the top stacking position, it returns to its original position rather than to the bottom.

6.8 The document must have a small image of yourself, which must appear when the mouse button is clicked at the position of the mouse cursor, regardless of the position of the cursor at the time.

6.9 The document must contain the statement "Save time with TIMESAVER 2.2," which continuously moves back and forth across the top of the display.

6.10 Modify the document of Exercise 6.9 to make the statement change color between red and blue every fifth step of its movement (assuming each move is 1 pixel long).

6.11 Modify the `mover` example in Section 6.10 to input the starting and ending position of the element to be moved.

CHAPTER

7

Introduction to XML

7.1 Introduction
7.2 The Syntax of XML
7.3 XML Document Structure
7.4 Document Type Definitions
7.5 Namespaces
7.6 XML Schemas
7.7 Displaying Raw XML Documents
7.8 Displaying XML Documents with CSS
7.9 XSLT Style Sheets
7.10 XML Processors
7.11 Web Services
Summary • Review Questions • Exercises

Some people consider the eXtensible Markup Language (XML) to be one of the most important among the parade of technologies developed to support the World Wide Web. Clearly, it has already had far-reaching effects on the storage and processing of data. XML consists of a collection of related technologies specified by recommendations developed by the W3C. This chapter provides introductions to the most important of these.

The chapter begins with a brief discussion of the origins of XML followed by a description of some of its characteristics. Then the general syntactic structure of XML documents is described. Next, the chapter details the purpose and form of document type definitions (DTDs), including the declarations of elements, attributes, and entities. A DTD provides the elements and attributes for

a markup language, as well as the rules for how the elements can appear in documents. This is followed by a description of XML namespaces. Next, XML schema is introduced. XML schema provides a more elaborate way to describe the structure of XML documents than DTDs. Two different approaches to formatting XML documents, CSS and XSLT style sheets, are then discussed and illustrated with examples. Actually, XSLT style sheets are used to transform XML documents. The target of the transformations we describe is an XHTML document, which can include CSS style specifications for display. Finally, we discuss the issues associated with reading and processing XML documents. Keep in mind that this chapter describes only a small part of XML and its associated technologies.

7.1 Introduction

A meta-markup language is a language for defining markup languages. The Standard Generalized Markup Language (SGML) is a meta-markup language for defining markup languages that can describe a wide variety of document types. In 1986 SGML was approved as an International Standards Organization (ISO) standard. In 1990 SGML was used as the basis for the development of HTML as the standard markup language for Web documents. In 1996 the World Wide Web Consortium (W3C) began work on XML, another meta-markup language. The first XML standard, 1.0, was published in February 1998.

Part of the motivation for the development of XML was the deficiencies of HTML. The purpose of HTML is to describe the layout of information in Web documents. To allow this, HTML defines a collection of tags and attributes. An HTML user can use that set of tags and attributes only. One problem with HTML is that it was defined to describe the layout of information without considering its meaning. So, regardless of the kind of information you are trying to describe with HTML, all you can really describe is its general form and layout in a document. For example, suppose that a document stores a list of used cars for sale, and the color and price are included for each car. With HTML, those two pieces of information about a car could be stored as the content of paragraph elements, but there would be no way to find them in the document because paragraph tags could have been used for many different kinds of information. To describe a particular kind of information, it would be necessary to have tags indicating the meaning of the element's content. That would allow processing of specific categories of information in a document. For example, if the price of a used car is stored as the content of an element named `price`, an application could find all cars in the document that cost less than $20,000. Of course, no markup language could possibly include meaningful tags for all of the different kinds of information that might be stored in documents.

Another potential problem with HTML is that it enforces few restrictions on the arrangement or order of tags in a document. For example, an opening tag can appear in the content of an element, but its corresponding closing tag

can appear after the end of the element in which it is nested. An example of this is as follows:

```
<strong> Now <em> is </strong> the time </em>
```

Note that although this problem was evident in HTML 4, which was in use when XML was developed, it is not a problem with XHTML, for obvious reasons.

One solution to the deficiencies of HTML is for each group of users with common document needs to develop its own set of tags and attributes and then use the SGML standard to define a new markup language to meet those needs. Each application area would have its own markup language. The problem with this is that SGML is too large and complex to make this approach feasible. SGML includes a large number of capabilities that are only rarely used. A program capable of parsing SGML documents would be very large and costly to develop. In addition, SGML requires that a formal definition be provided with each new markup language. So, although having area-specific markup languages is a good idea, basing them on SGML is not.

An alternative solution to the problems of HTML is to define a simplified version of SGML and allow users to define their own markup languages based on it. XML was designed to be that simplified version of SGML. In this context, "users" means organizations of people with common data description and processing needs (rather than individual users). For example, chemists need to store chemical data in a standard way, providing a way to share data with other chemists and allowing all to use data processing tools that work on chemical data stored in the same standard format, regardless of its origin. Likewise, this is the case for many other groups with their own kinds of data to represent and process.

It is important to understand that XML was not meant to be a replacement for HTML. In fact, the two have different goals. Whereas HTML is a markup language that is meant to describe the layout of general information, as well as provide some guidance for how it should be displayed, XML is a meta-markup language that provides a framework for defining specialized markup languages. HTML itself can be defined as an XML markup language. In fact, XHTML is an XML-based version of HTML.

XML is far more than a solution to the deficiencies of HTML. It provides a simple and universal way of storing any textual data. Data stored in XML documents can be electronically distributed and processed by any number of different processing applications. These applications are relatively easy to write because of the standard ways in which the data is stored. Therefore, XML is a universal data interchange language.

XML is not a markup language; it is a meta-markup language that specifies rules for creating markup languages. As a result, XML includes no tags. When designing a markup language using XML, the designer must define a collection of tags that carries the meaning of the content of the tags. As with XHTML, an XML tag and its content, together with the closing tag, are called an *element*.

Strictly speaking, a markup language designed with XML is called an *XML application*. However, a program that processes information stored in a docu-

ment formatted with an XML application is also called an application. To avoid confusion, we will refer to an XML-based markup language as a *tag set*. We call documents that use an XML-based markup language *XML documents*.

XML documents can be written by hand using a simple text editor. This approach is, of course, impractical for large data collections, which are likely to be written by programs. A browser has a default presentation style for every XHTML element, which makes it possible for the browser to display any XHTML document. However, a browser cannot be expected to have default presentation styles for elements it has never seen. Therefore, the data in an XML document can be displayed by browsers only if the presentation styles are provided by style sheets of some kind.

Application programs that process the data in XML documents must analyze the document before they gain access to the data. This analysis is performed by an XML processor, which has several tasks, one of which is to parse XML documents, a process that isolates the constituent parts (such as tags, attributes, and data strings) and provides them to an application. XML processors are described in Section 7.10.

Unlike most documents produced by word processing systems, XML documents have no hidden specifications. Therefore, XML documents are plain text, which is easily readable by both people and application programs (although there are no compelling reasons for people to read them).

At the time of this writing (early 2007), the vast majority of Web clients use either Internet Explorer 6 (IE6), Internet Explorer 7 (IE7), Firefox (FX), or Firefox 2 (FX2) browsers, all of which support basic XML.

7.2 The Syntax of XML

The syntax of XML can be thought of at two distinct levels. First, there is the general low-level syntax of XML that imposes its rules on all XML documents. The other syntactic level is specified by either document type definitions (DTDs) or XML schemas. These two kinds of specifications impose structural syntactic rules on documents written with specific XML tag sets. DTDs and XML schemas specify the set of tags and attributes that can appear in a particular document or collection of documents, and also the orders and various arrangements in which they can appear. So, either a DTD or an XML schema can be used to define an XML-based markup language. DTDs are described in Section 7.4. XML schemas are discussed in Section 7.6. This section describes the first level of XML syntax, that which applies to all XML documents.

An XML document can include several different kinds of statements. The most common of these are the data elements of the document. XML documents may also include markup declarations, which are instructions to the XML parser, and processing instructions, which are instructions for an application program that will process the data described in the document.

All XML documents begin with an XML declaration, which has the appearance of a processing instruction but technically is not one. The XML declara-

tion identifies the document as being XML and provides the version number of the XML standard being used. It may also specify an encoding standard. The XML declaration appears as the first line of all XHTML documents in this book.

Comments in XML are the same as in HTML. They cannot contain two adjacent dashes, for obvious reasons.

XML names are used to name elements and attributes. An XML name must begin with a letter or an underscore and can include digits, hyphens, and periods. XML names are case sensitive, so `Body`, `body`, and `BODY` are all distinct names. There is no length limitation for XML names.

A small set of syntax rules applies to all XML documents. XHTML uses the same rules, and the XHTML markup in this book complies with them.

Every XML document defines a single root element, whose opening tag must appear on the first line of XML code. All other elements of an XML document must be nested inside the root element. The root element of every XHTML document is `html`. XML tags, like those of XHTML, are surrounded by angle brackets.

Every XML element that can have content must have a closing tag. Elements that do not include content must use a tag with the following form:

<element_name />

As is the case with XHTML, XML tags can have attributes, which are specified with name/value assignments. As with XHTML, all attribute values must be enclosed by either single or double quotation marks.

An XML document that strictly adheres to these syntax rules is considered *well formed*. Consider the following simple but complete example:

```
<?xml version = "1.0" encoding = "utf-8"?>
<ad>
   <year> 1960 </year>
   <make> Cessna </make>
   <model> Centurian </model>
   <color> Yellow with white trim </color>
   <location>
      <city> Gulfport </city>
      <state> Mississippi </state>
   </location>
</ad>
```

Notice that none of the tags in this document is defined in XHTML—all are designed for the specific content of the document. This document effectively defines an XML tag set. This illustrates that an XML-based markup language can be defined without a DTD or an XML schema, although it is an informal definition with no structure rules.

When designing an XML document, the designer is often faced with the choice between adding a new attribute to an element or defining a nested element. In some cases, there is no choice. For example, if the data in question is

an image, a reference to it can only be an attribute because such a reference cannot be the content of an element (because images are binary data and XML documents can contain only text). In other cases, it may not matter whether an attribute or a nested element is used. However, there are some situations in which there is a choice and one is clearly better than the other.

In some situations, nested tags are better than attributes. A document or category of documents for which you are defining tags might need to grow in structural complexity in the future. Nested tags can be added to any existing tag to describe its growing size and complexity. Nothing can be added to an attribute, however. Attributes cannot describe structure at all, so a nested element should be used if the data in question has some substructure of its own. A nested element should be used if the data is subdata of the parent element's content rather than information about the data of the parent element.

There is one situation in which an attribute should always be used: for identifying numbers or names of elements, exactly as the `id` and `name` attributes are used in XHTML. An attribute also should be used if the data in question is one value from a given set of possibilities. Finally, attributes should be used if there is no substructure or if it is really just information about the element.

The following versions of an element named `patient` illustrate three possible choices between tags and attributes:

```
<!-- A tag with one attribute -->
<patient name = "Maggie Dee Magpie">
   ...
</patient>

<!-- A tag with one nested tag -->
<patient>
   <name> Maggie Dee Magpie </name>
   ...
</patient>

<!-- A tag with one nested tag, which contains
      three nested tags -->
<patient>
   <name>
      <first> Maggie </first>
      <middle> Dee </middle>
      <last> Magpie </last>
   </name>
   ...
</patient>
```

In this example, the third choice is probably the best because it provides easy access to all of the parts of the data, which may be needed. Also, there is no compelling reason to use attributes in this structure.

7.3 XML Document Structure

An XML document often uses two auxiliary files: one that specifies its tag set and structural syntactic rules and one that contains a style sheet to describe how the content of the document is to be printed or displayed. The structural syntactic rules are given as either a DTD or an XML schema. Two approaches to style specification are discussed in Sections 7.8 and 7.9.

An XML document consists of one or more entities that are logically related collections of information, ranging in size from a single character to a book chapter. One of these entities, called the *document entity*, is always physically in the file that represents the document. The document entity can be the entire document, but in many cases it includes references to the names of entities that are stored elsewhere. For example, the document entity for a technical article might contain the beginning material and ending material but have references to the article body sections, which are entities stored in separate files. Every entity except the document entity must have a name.

There are several reasons to break a document into multiple entities. It is good to define a large document as a number of smaller parts to make it more manageable. Also, if the same data appears in more than one place in the document, defining it as an entity allows any number of references to a single copy of the data. This avoids the problem of inconsistency among the occurrences. Finally, many documents include information that cannot be represented as text, such as images. Such information units are usually stored as binary data. If a binary data unit is logically part of a document, it must be a separate entity because XML documents cannot include binary data. Such entities are called *binary entities*.

When an XML processor encounters the name of a nonbinary entity in a document, it replaces the name with the value it references. Binary entities can be handled only by applications that deal with the document, such as browsers. XML processors deal only with text.

Entity names can be any length. They must begin with a letter, a dash, or a colon. After the first character, a name can have letters, digits, periods, dashes, underscores, or colons. A reference to an entity is its name with a prepended ampersand and an appended semicolon. For example, if `apple_image` is the name of an entity, `&apple_image;` is a reference to it.

One of the common uses of entities is to allow characters that are normally used as markup delimiters to appear as themselves in a document. Because this is a common need, XML includes the entities that are predefined for XHTML, the most common of which are shown in Table 2.1 (in Chapter 2). User-defined entities can be defined only in DTDs, which are discussed in Section 7.4.

When several predefined entities must appear near each other in an XML document, their references clutter the content and make it difficult to read. In such cases, a character data section can be used. The content of a character data section is not parsed by the XML parser, so it cannot include any tags. This promise of no tags makes it possible to include special markup delimiter charac-

ters directly in the section without using their entity references. The form of a character data section is as follows:

```
<![CDATA[ content ]]>
```

For example, instead of

```
The last word of the line is &gt;&gt;&gt; here
&lt;&lt;&lt;.
```

the following could be used:

```
<![CDATA[The last word of the line is >>> here <<<]]>
```

The opening keyword of a character data section is not just `CDATA`; it is in effect `[CDATA[`. An important consequence of this is that there cannot be any spaces between the `[` and the `C`, or between the `A` (the last character of `CDATA`) and the second `[`.

Because the content of a character data section is not parsed by the XML parser, any entity references that are included are not expanded. For example, the content of the line

```
<![CDATA[The form of a tag is &lt;tag name&gt;]]>
```

is as follows:

```
The form of a tag is &lt;tag name&gt;
```

7.4 Document Type Definitions

A document type definition (DTD) is a set of structural rules called *declarations*, which specify a set of elements that can appear in the document as well as how and where these elements may appear. DTDs also provide entity definitions. Not all XML documents need a DTD. Use of a DTD is related to the use of an external style sheet for XHTML documents. External style sheets are used to impose a uniform style over a collection of documents. DTDs are used when the same tag set definition is used by a collection of documents, perhaps by a collection of users, and the documents must have a consistent and uniform structure.

A document can be tested against the DTD to determine whether it conforms to the rules the DTD describes. Application programs that process the data in the collection of XML documents can be written to assume the particular document form. Without such structural restrictions, developing such applications would be difficult, if not impossible.

A DTD can be embedded in the XML document whose syntax rules it describes, in which case it is called an *internal DTD*. The alternative is to have the DTD stored in a separate file, in which case it is called an *external DTD*. Because external DTDs allow use with more than one XML document, they are preferable. A group of users defines a DTD for their particular kind of data and they all use that DTD, which imposes structural uniformity across all of their documents.

It is common knowledge that the earlier errors in software systems are found, the less expensive it is to fix them. The situation is similar in the case of DTDs. A DTD with an incorrect or inappropriate declaration can have widespread consequences. Fixing the DTD and all copies of it is just the first step, and it is the simplest. After the correction of the DTD is completed, all documents that use the DTD must be tested against the DTD and often modified to conform to the changed DTD. Changes to associated style sheets also might be necessary.

Syntactically, a DTD is a sequence of declarations. Each declaration has the form of a markup declaration:

```
<!keyword ... >
```

Four possible keywords can be used in a declaration: `ELEMENT`, used to define tags; `ATTLIST`, used to define tag attributes; `ENTITY`, used to define entities; and `NOTATION`, used to define data type notations. The first three of these kinds of declarations are described in the following sections. Because of their infrequent use, we do not discuss `NOTATION` declarations.

7.4.1 Declaring Elements

The element declarations of a DTD have a form that is related to that of the rules of context-free grammars, also known as Backus-Naur form (BNF).[1] BNF is used to define the syntactic structure of programming languages. A DTD describes the syntactic structure of a particular set of documents, so it is natural for its rules to be similar to those of BNF.

Each element declaration in a DTD specifies the structure of one category of elements. The declaration provides the name of the element whose structure is being defined, along with the specification of the structure of that element. Although an XML document actually is a string of characters, it is often convenient to think of it in terms of a general tree. An element is a node in such a tree, either a leaf node or an internal node. If the element is a leaf node, its syntactic description is its character pattern. If the element is an internal node, its syntactic description is a list of its child elements, each of which can be a leaf node or an internal node.

The form of an element declaration for elements that contain elements is as follows:

`<!ELEMENT` *element_name* `(`*list of names of child elements*`)>`

For example, consider the following declaration:

```
<!ELEMENT memo (from, to, date, re, body)>
```

This element declaration would describe the document tree structure shown in Figure 7.1.

1. BNF is named after its primary designer, John Backus, and Peter Naur, who helped by providing some small modifications.

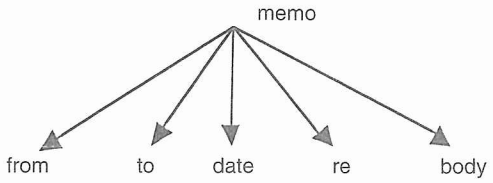

Figure 7.1 An example of the document tree structure for an element definition

In many cases, it is necessary to specify the number of times that a child element may appear. This can be done in a DTD declaration by adding a modifier to the child element specification. These modifiers, described in Table 7.1, are borrowed from regular expressions. Any child element specification can be followed by one of the modifiers.

Table 7.1 Child element specification modifiers

Modifier	Meaning
+	One or more occurrences
*	Zero or more occurrences
?	Zero or one occurrence

Consider the following DTD declaration:

```
<!ELEMENT person (parent+, age, spouse?, sibling*)>
```

In this example, a `person` element is specified to have the following children elements: one or more `parent` elements, one `age` element, possibly a `spouse` element, and zero or more `sibling` elements.

The leaf nodes of a DTD specify the data types of the content of their parent nodes, which are elements. In most cases, the content of an element is type `PCDATA`, for *p*arsable *c*haracter *data*. Parsable character data is a string of any printable characters except less-than (<) and ampersand (&). Two other content types can be specified: `EMPTY` and `ANY`. The `EMPTY` type is used to specify that the element has no content. This is used for elements similar to the XHTML `img` element. The `ANY` type is used when the element may contain literally any content. The form of a leaf element declaration is as follows:

```
<!ELEMENT element_name (#PCDATA)>
```

7.4.2 Declaring Attributes

The attributes of an element are declared separately from the element declaration in a DTD. An attribute declaration must include the name of the element to which the attribute belongs, the attribute's name, and its type. Also, it may include a default value. The general form of an attribute declaration is as follows:

<!ATTLIST *element_name attribute_name attribute type* [*default_value*]>

If more than one attribute is declared for a given element, the declarations can be combined, as in the following example:

```
<!ATTLIST element_name
          attribute name_1 attribute type default_value_1
          attribute_name_2 attribute_type default_value_2
          ...
          attribute_name_n attribute_type default_value_n
>
```

There are ten different attribute types. For this chapter, only one, CDATA, will be used. This type is just any string of characters that does not include less-than and ampersand characters.

The default value in an attribute declaration can specify either an actual value or a requirement for the value of the attribute in the XML document. Table 7.2 lists the possible default values.

Table 7.2 Possible default values for attributes

Value	Meaning
A value	The quoted value, which is used if none is specified in an element
#FIXED value	The quoted value, which every element will have and which cannot be changed.
#REQUIRED	No default value is given; every instance of the element must specify a value
#IMPLIED	No default value is given (the browser chooses the default value); the value may or may not be specified in an element

For example, suppose the DTD included the following attribute specifications:

```
<!ATTLIST airplane places CDATA "4">
<!ATTLIST airplane engine_type CDATA #REQUIRED>
<!ATTLIST airplane price CDATA #IMPLIED>
<!ATTLIST airplane manufacturer CDATA #FIXED "Cessna">
```

The following XML element is valid for this DTD:

```
<airplane places = "10" engine_type = "jet"> </airplane>
```

Attributes that include `#FIXED` in the DTD may or may not be specified in particular element instances.

7.4.3 Declaring Entities

Entities can be defined so that they can be referenced anywhere in the content of an XML document, in which case they are called *general entities*. The predefined entities are all general entities. Entities can also be defined so that they can be referenced only in DTDs, in which case they are called *parameter entities*.

The form of an entity declaration that appears in a DTD follows:

```
<!ENTITY [%] entity_name "entity_value">
```

When the optional percent sign (`%`) is present in an entity declaration, it specifies that the entity is a parameter entity rather than a general entity.

Consider the following example of an entity. Suppose that a document includes a large number of references to the full name of President Kennedy. You could define an entity to represent his complete name as follows:

```
<!ENTITY jfk "John Fitzgerald Kennedy">
```

Any XML document that uses a DTD that includes this declaration can specify the complete name with just the reference `&jfk;`.

When an entity is longer than a few words, such as a section of a technical article, its text is defined outside the DTD. In such cases, the entity it is called an *external text entity*. The form of the declaration of an external text entity follows:

```
<!ENTITY entity_name SYSTEM "file_location">
```

The keyword `SYSTEM` specifies that the definition of the entity is in a different file, which is specified as the string following `SYSTEM`.

7.4.4 A Sample DTD

As an example of a DTD, consider a booklet of ads for used aircraft. In this case, the DTD describes the form of the booklet and each of its ads:

```
<?xml version = "1.0" encoding = "utf-8"?>

<!-- planes.dtd - a document type definition for
                  the planes.xml document, which specifies
                  a list of used airplanes for sale  -->

<!ELEMENT planes_for_sale (ad+)>
<!ELEMENT ad (year, make, model, color, description,
```

```
              price?, seller, location)>
<!ELEMENT year (#PCDATA)>
<!ELEMENT make (#PCDATA)>
<!ELEMENT model (#PCDATA)>
<!ELEMENT color (#PCDATA)>
<!ELEMENT description (#PCDATA)>
<!ELEMENT price (#PCDATA)>
<!ELEMENT seller (#PCDATA)>
<!ELEMENT location (city, state)>
<!ELEMENT city (#PCDATA)>
<!ELEMENT state (#PCDATA)>

<!ATTLIST seller phone CDATA #REQUIRED>
<!ATTLIST seller email CDATA #IMPLIED>

<!ENTITY c "Cessna">
<!ENTITY p "Piper">
<!ENTITY b "Beechcraft">
```

Some XML parsers check documents that have DTDs to ensure that the documents conform to the structure specified in the DTDs. These parsers are called *validating parsers*. Not all XML parsers are validating parsers. If an XML document specifies a DTD and is parsed by a validating XML parser, and the parser determines that the document conforms to the DTD, the document is called *valid*.

Handwritten XML documents often are not well formed, which means that they do not follow XML's syntactic rules. These errors are detected by all XML parsers, which must report them. Because errors are common, it is important to check that XML documents are well-formed before making them available to site visitors. XML parsers are not allowed to either repair or ignore errors. Validating XML parsers detect and report all inconsistencies in documents relative to their DTDs. XML parsers are discussed in Section 7.10.

7.4.5 Internal and External DTDs

Recall that a DTD can appear inside an XML document or in an external file, as is the case with `planes.dtd`. If the DTD is included in the XML code, it must be introduced with `<!DOCTYPE` *root_name* `[` and terminated with `]>`. For example, the structure of the `planes` XML document with its DTD included is as follows:

```
<?xml version = "1.0" encoding = "utf-8"?>
      <!DOCTYPE planes [
          <!-- The DTD for planes -->
      ]>
<!-- The planes XML document -->
```

When the DTD is in its own file, the XML document refers to it with a DOCTYPE declaration as its second line. This declaration has the following form:

<!DOCTYPE *XML_document_root_name* SYSTEM "*DTD_file_name*">

For the planes example, assuming that the DTD is stored in the file named planes.dtd, this declaration would be as follows:

```
<!DOCTYPE planes_for_sale SYSTEM "planes.dtd">
```

The following is an example of an XML document that is valid for the planes DTD:

```
<?xml version = "1.0" encoding = "utf-8"?>

<!-- planes.xml - A document that lists ads for
                  used airplanes -->

<!DOCTYPE planes_for_sale SYSTEM "planes.dtd">
<planes_for_sale>
   <ad>
      <year> 1977 </year>
      <make> &c; </make>
      <model> Skyhawk </model>
      <color> Light blue and white </color>
      <description> New paint, nearly new interior,
            685 hours SMOH, full IFR King avionics </description>
      <price> 23,495 </price>
      <seller phone = "555-222-3333"> Skyway Aircraft </seller>
      <location>
         <city> Rapid City, </city>
         <state> South Dakota </state>
      </location>
   </ad>
   <ad>
      <year> 1965 </year>
      <make> &p; </make>
      <model> Cherokee </model>
      <color> Gold </color>
      <description> 240 hours SMOH, dual NAVCOMs, DME,
                new Cleveland brakes, great shape </description>
      <seller phone = "555-333-2222"
              email = "jseller@www.axl.com">
              John Seller </seller>
      <location>
```

```
        <city> St. Joseph, </city>
        <state> Missouri </state>
      </location>
   </ad>
</planes_for_sale>
```

7.5 Namespaces

It is often convenient to construct XML documents that include tag sets that are defined for and used by other documents. When a tag set is available and appropriate for a particular XML document or class of documents, it is better to use it than to invent a new collection of element types. For example, imagine that you must define an XML markup language for a furniture catalog with `<chair>`, `<sofa>`, and `<table>` tags. The catalog document must also include several different tables of specific furniture pieces, wood types, finishes, and prices. It is convenient to use XHTML table tags to define these tables rather than inventing a new vocabulary for them.

The obvious problem with using different markup vocabularies in the same document is that collisions between names that are defined in two or more of those tag sets could result. An example of this is having a `<table>` tag for a category of furniture and a `<table>` tag from XHTML for information tables. Clearly, software systems that process XML documents must be capable of unambiguously recognizing the element names in those documents. To deal with this problem, the W3C has developed a standard for XML namespaces at (`http://www.w3.org/TR/REC-xml-names`).

An *XML namespace* is a collection of element and attribute names used in XML documents. The name of a namespace usually has the form of a uniform resource identifier (URI).[2] A namespace for the elements and attributes of the hierarchy rooted at a particular element is declared as the value of the attribute `xmlns`. The form of a namespace declaration for an element follows:

<*element_name* `xmlns[:`*prefix*`]` = URI>

The square brackets indicate that what is within them is optional. The prefix, if included, is the name that must be attached to the names in the declared namespace. A prefix is used for two reasons. First, the URI is too long to be typed on every occurrence of every name from the namespace. Second, a URI includes characters that are illegal in XML. Note that the element for which a namespace is declared is usually the root of a document. For example, all XHTML documents in this book declare the `xmlns` namespace on the root element:

```
<html xmlns = "http://www.w3.org/1999/xhtml">
```

2. A URL is a URI that happens to be the Internet address of some resource.

This declares the default namespace, from which names may appear without prefixes.

As an example of a prefixed namespace declaration, consider the following:

```
<birds  xmlns:bd = "http://www.audubon.org/names/species">
```

Within the `birds` element, including all of its children elements, the names from the namespace must be prefixed with `bd`, as in the following:

```
<bd:lark>
```

If an element has more than one namespace declaration, these are declared as follows:

```
<birds  xmlns:bd = "http://www.audubon.org/names/species"
        xmlns:html = "http://www.w3.org/1999/xhtml" >
```

In this example, we have added the standard XHTML namespace to the `birds` element. Note that in any namespace declaration one of the namespaces can be specified as the default by omitting the prefix.

One namespace declaration in an element can be used to declare a default namespace. This is done by simply not including the prefix in the declaration. The names from the default namespace can be used without a prefix.

Consider the following example in which two namespaces are declared. The first is declared to be the default namespace; the second defines the prefix, `cap`.

```
<states>
  xmlns = "http://www.states-info.org/states"
  xmlns:cap = "http://www.states-info.org/state-capitals"
  <state>
     <name> South Dakota </name>
     <population> 754844 </population>
     <capital>
        <cap:name> Pierre </cap:name>
        <cap:population> 12429 </cap:population>
     </capital>
  </state>
  <!-- More states -->
</states>
```

Each state element has name and population elements from both namespaces.

Note that attribute names are not included in namespaces because attribute names are local to elements, so a tag set may use the same attribute name in more than one element without causing ambiguity.

If an XML document uses a DTD and a prefixed name, the DTD must define an element with exactly the same prefix and name.

Because of their form, it is tempting to think that a namespace is a Web resource that lists element names. But that is never the case. The standard

namespaces (e.g., `http://www.w3.org/1999/xhtml`) often are valid URLs, but they are documents that describe far more than a set of element names. User-defined namespace names do not need to use the URI form, although that is a good way to prevent conflicts with namespace names.

7.6 XML Schemas

DTDs have several disadvantages. One is that DTDs are written in a syntax unrelated to XML, so they cannot be analyzed with an XML processor. Also, it can be confusing for people to deal with two different syntactic forms, one to define a document and one to define its structure. Another disadvantage is that DTDs do not allow restrictions on the form of data that can be the content of a particular tag. For example, if the content of an element represents time, regardless of the form of the time data, a DTD can only specify that it is text, which could be anything. In fact, the content of an element could be an integer number, a floating-point number, or a range of numbers. All of these would be specified as text. With DTDs, there are only ten data types, none of which is numeric.

Several alternatives to DTDs have been developed to attempt to overcome their weaknesses. XML schema, which was designed by the W3C, is one of these alternatives. We have chosen to discuss it because of its W3C support and the likelihood that it will become the primary successor to the DTD-based system. An XML schema is an XML document, so it can be parsed with an XML parser. It also provides far more control over data types than do DTDs. The content of a specific element can be required to be any one of 44 different data types. Furthermore, the user can define new types with constraints on existing data types. For example, a numeric data value can be required to have exactly seven digits.

To promote the transition from DTDs to XML schemas, XML schema was designed to allow any DTD to be automatically converted to an equivalent XML schema.

7.6.1 Schema Fundamentals

Schemas can conveniently be related to the idea of a class and an object in an object-oriented programming language. A schema is similar to a class definition; an XML document that conforms to the structure defined in the schema is similar to an object of the schema's class. In fact, XML documents that conform to a specific schema are considered instances of that schema.

Schemas have two primary purposes. First, a schema specifies the structure of its instance XML documents, including which elements and attributes may appear in the instance document, as well as where and how often they may appear. Second, a schema specifies the data type of every element and attribute

of its instance XML documents. This is the area in which schemas far outshine DTDs.

It has been said that XML schemas are "namespace centric." There is some truth to that depiction. In XML schemas, as in XML, namespaces are represented by names that have the form of URIs. Because they must be unique, it is customary to use URIs that start with the author's Web site address for namespaces. For example, for namespaces used in this section we use the prefix `"http://cs.uccs.edu/"`. To this we add whatever name is connotative of the specific application.

7.6.2 Defining a Schema

Schemas themselves are written using a collection of names, or a vocabulary, from a namespace that is, in effect, a schema of schemas. The name of this namespace is `http://www.w3.org/2001/XMLSchema`. Some of the names in this namespace are `element`, `schema`, `sequence`, and `string`.

Every schema has `schema` as its root element. As stated, the `schema` element specifies the namespace for the schema of schemas from which the schema's elements and attributes will be drawn. It often also specifies a prefix that will be used for the names in the schema. This namespace specification appears as follows:

```
xmlns:xsd = "http://www.w3.org/2001/XMLSchema"
```

This provides the prefix `xsd` for the names from the namespace for the schema of schemas.

A schema defines a namespace in the same sense as a DTD defines a tag set. The name of the namespace defined by a schema must be specified with the `targetNamespace` attribute of the `schema` element. Every top-level (not nested) element that appears in a schema places its name in the target namespace. The target namespace is specified by assigning a namespace to the target namespace attribute, as in the following:

```
targetNamespace = "http://cs.uccs.edu/planeSchema"
```

If we want the elements and attributes that are not defined directly in the schema element (they are nested inside top-level elements) to be included in the target namespace, schema's `elementFormDefault` must be set to `qualified`, as in the following:

```
elementFormDefault = "qualified"
```

The default namespace, which is the source of the unprefixed names in the schema, is given with another `xmlns` specification, but this time without the prefix. For example:

```
xmlns = "http://cs.uccs.edu/planeSchema"
```

An example of a complete opening tag for a schema is as follows:

```
    <xsd:schema
<!-- The namespace for the schema itself (prefix is xsd) -->
       xmlns:xsd = http://www.w3.org/2001/XMLSchema
<!-- The namespace where elements defined here will be placed -->
       targetNamespace = http://cs.uccs.edu/planeSchema
<!-- The default namespace for this document (no prefix) -->
       xmlns = http://cs.uccs.edu/planeSchema
<!-- We want to put non-top-level elements in the target namespace -->
       elementFormDefault = "qualified">
```

In this example, the target namespace and the default namespace are the same.

One alternative to the preceding opening tag would be to make the `XMLSchema` names the default so that they do not need to be prefixed in the schema. Then the names in the target namespace would need to be prefixed. The following schema tag illustrates this:

```
<schema
   xmlns = "http://www.w3.org/2001/XMLSchema"
   targetNamespace = "http://cs.uccs.edu/planeSchema"
   xmlns:plane = "http://cs.uccs.edu/planeSchema"
   elementFormDefault = "qualified">
```

Notice that the name `schema` in this tag name does not need to be prefixed because its namespace is now the default. However, all of the names being created by this schema must be prefixed, both in the schema and in its instances.

7.6.3 Defining a Schema Instance

An instance of a schema must include specifications of the namespaces it uses. These are given as attribute assignments in the tag for its root element. First, an instance document normally defines its default namespace to be the one defined in its schema. For example, if the root element is `planes`, we could have the following:

```
<planes
   xmlns = http://cs.uccs.edu/planeSchema
   ... >
```

The second attribute specification in the root element of an instance document is for the `schemaLocation` attribute. This attribute is used to name the standard namespace for instances, which is `XMLSchema-instance`. This namespace corresponds to the `XMLSchema` namespace used for schemas. The following attribute assignment specifies the `XMLSchema-instance` namespace and defines the prefix, `xsi`, for it:

```
xmlns:xsi = "http://www.w3.org/2001/XMLSchema-instance"
```

Third, the instance document must specify the filename of the schema where the default namespace is defined. This is accomplished with the `schemaLocation` attribute, which takes two values: the namespace of the schema and the filename of the schema. This attribute is defined in the `XMLSchema-instance` namespace, so it must be named with the proper prefix. For example:

```
xsi:schemaLocation = "http://cs.uccs.edu/planeSchema
                      planes.xsd"
```

This is a peculiar attribute assignment in that it assigns two values, which are separated only by whitespace.

Altogether, the opening root tag of an XML instance of the `planes.xsd` schema, where the root element name in the instance is `planes`, could appear as follows:

```
<planes
   xmlns = "http://cs.uccs.edu/planeSchema"
   xmlns:xsi = "http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation = "http://cs.uccs.edu/planeSchema
                         planes.xsd">
```

The purpose of both DTDs and XML schemas is to provide a technique for standardization of the tag set and structure of families of XML documents. Conformance checking of an XML document against an XML schema can be done with any one of several available validation programs. One of these, named `xsv`, is discussed in Section 7.6.7. An XML schema validation program performs two kinds of conformance checks: First, it checks to determine whether the schema is valid relative to the schema of schemas, `XMLSchema`. Second, it checks to determine whether the XML document conforms to the syntactic rules specified in the schema of which the document is an instance.

7.6.4 An Overview of Data Types

There are two categories of user-defined XML schema data types: simple and complex. A *simple data type* is one whose content is restricted to strings. A simple type cannot have attributes or include nested elements. The string restriction seems like it would make simple types a very narrow type category, but in fact it does not because a large collection of predefined data types are included in the category. Some of these are mentioned in this section. A *complex type* can have attributes and include other data types as elements.

XML schema defines 44 data types, 19 of which are primitive and 25 of which are derived. The primitive data types include `string`, `Boolean`, `float`, `time`, and `anyURI`. The predefined derived types include `byte`, `long`, `decimal`, `unsignedInt`, `positiveInteger`, and `NMTOKEN`. User-defined data types are defined by specifying restrictions on an existing type, which is then called a *base type*. Such user-defined types are derived types. Constraints in derived types are given in terms of the *facets* of the base type. For example, the

integer primitive data type has eight possible facets: totalDigits, maxInclusive, maxExclusive, minInclusive, minExclusive, pattern, enumeration, and whitespace. Examples of user-defined data types are given in Section 7.6.5. A list of all predefined data types can be found at http://www.w3.org/TR/xmlschema-2/#built-in-datatypes.

Both simple and complex types can be *named* or *anonymous*. If anonymous, a type cannot be used outside the element in which it is declared.

Elements in a DTD are all global. Each has a unique name and is defined exactly once. The context of a reference to a DTD element is irrelevant. By contrast, context is essential to defining the meaning of a reference to an element in an XML schema.

Data declarations in an XML schema can be either local or global. A *local declaration* is one that appears inside an element that is a child of the schema element; that is, a declaration in a grandchild element of schema (or a more distant descendant) is a local declaration. A locally declared element is visible only in that element. This means that local elements with the same name can appear in any number of different elements with no interference among them. A *global declaration* is one that appears as a child of the schema element. Global elements are visible in the whole schema in which they are declared.

7.6.5 Simple Types

Elements are defined in an XML schema with the element tag, which is from the XMLSchema namespace. Recall that the prefix xsd is normally used for names from this namespace. An element that is named includes the name attribute for that purpose. The other attribute that is necessary in a simple element declaration is type, which is used to specify the type of content allowed in the element. For example:

```
<xsd:element name = "engine"  type = "xsd:string" />
```

An instance of the schema in which the engine element is defined could have the following element:

```
<engine> inline six cylinder fuel injected </engine>
```

An element can be given a default value using the default attribute. For example:

```
<xsd:element name = "engine"   type = "xsd:string"
             default = "fuel injected V-6"  />
```

Elements can have constant values, meaning that the content of the defined element in every instance document has the same value. Constant values are given with the fixed attribute, as in the following example:

```
<xsd:element name = "plane"   type = "xsd:string"
             fixed = "single wing"  />
```

We now turn our attention to user-defined data types, which are constrained predefined types. A simple user-defined data type is described in a `simpleType` element, using facets. Facets must be specified in the content of a `restriction` element, which gives the base type name. The facets themselves are given in elements named for the facets, using the `value` attribute to specify the value of the facet. For example, the following declares a user-defined type, `firstName`, for strings of fewer than 11 characters:

```
<xsd:simpleType name = "firstName">
    <xsd:restriction base = "xsd:string">
        <xsd:maxLength value = "10" />
    </xsd:restriction>
</xsd:simpleType>
```

The `length` facet is used to restrict the string to an exact number of characters. The `minLength` facet is used to specify a minimum length. The number of digits of a decimal number is restricted with the `precision` facet. For example:

```
<xsd:simpleType name = "phoneNumber">
    <xsd:restriction base = "xsd:decimal">
        <xsd:precision value = "7" />
    </xsd:restriction>
</xsd:simpleType>
```

7.6.6 Complex Types

Most XML documents include nested elements, so few XML schemas do not have complex types. Although there are several categories of complex element types, we restrict our discussion to those called *element-only elements*, which can have elements in their content but no text. All complex types can have attributes.

Complex types are defined with the `complexType` tag. The elements that are the content of an element-only element must be contained in an ordered group, an unordered group, a choice, or a named group. Ordered and unordered groups are discussed here.

The `sequence` element is used to contain an ordered group of elements. For example, consider the following type definition:

```
<xsd:complexType name = "sports_car">
   <xsd:sequence>
       <xsd:element name = "make"  type = "xsd:string" />
       <xsd:element name = "model"  type = "xsd:string" />
       <xsd:element name = "engine"  type = "xsd:string" />
       <xsd:element name = "year"  type = "xsd:decimal" />
    </xsd:sequence>
</xsd:complexType>
```

A complex type whose elements are an unordered group is defined in an `all` element.

Elements and `all` and `sequence` groups can include attributes to specify the numbers of occurrences. These attributes are `minOccurs` and `maxOccurs`. The possible values of `minOccurs` are the non-negative integers, including zero. The possible values for `maxOccurs` are the non-negative integers plus the value `unbounded`, which has the obvious meaning.

Consider the following complete example of a schema:

```
<?xml version = "1.0" encoding = "utf-8"?>

<!-- planes.xsd
     A simple schema for planes.xml
     -->
<xsd:schema
    xmlns:xsd = "http://www.w3.org/2001/XMLSchema"
    targetNamespace = "http://cs.uccs.edu/planeSchema"
    xmlns = "http://cs.uccs.edu/planeSchema"
    elementFormDefault = "qualified">

    <xsd:element name = "planes">
        <xsd:complexType>
            <xsd:all>
                <xsd:element name = "make"
                             type = "xsd:string"
                             minOccurs = "1"
                             maxOccurs = "unbounded" />
            </xsd:all>
        </xsd:complexType>
    </xsd:element>
</xsd:schema>
```

Notice that we use the `all` element to contain the single element of the complex type, `planes`. We could have used `sequence` instead. Because there is only one contained element, it makes no difference.

An XML instance that conforms to the `planes.xsd` schema follows:

```
<?xml version = "1.0" encoding = "utf-8"?>

<!-- planes.xml
     A simple XML document for illustrating a schema
     The schema is in planes.xsd
     -->
<planes
  xmlns = "http://cs.uccs.edu/planeSchema"
```

```
  xmlns:xsi = "http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation = "http://cs.uccs.edu/planeSchema
                        planes.xsd">
    <make> Cessna </make>
    <make> Piper </make>
    <make> Beechcraft </make>
</planes>
```

If we want the `year` element in the `sports_car` element that was defined earlier to be a derived type, we could define the derived type as another global element and refer to it in the `sports_car` element. For example, the `year` element could be defined as follows:

```
<xsd:element name = "year">
    <xsd:simpleType>
        <xsd:restriction base = "xsd:decimal">
            <xsd:minInclusive value = "1900" />
            <xsd:maxInclusive value = "2007" />
        </xsd:restriction>
    </xsd:simpleType>
</xsd:element>
```

With the `year` element defined globally, the `sports_car` element can be defined with a reference to the `year` with the `ref` attribute, as shown in the following:

```
<xsd:complexType name = "sports_car">
    <xsd:sequence>
       <xsd:element name = "make"  type = "xsd:string" />
       <xsd:element name = "model"  type = "xsd:string" />
       <xsd:element name = "engine"  type = "xsd:string" />
       <xsd:element ref = "year" />
    </xsd:sequence>
</xsd:complexType>
```

7.6.7 Validating Instances of Schemas

An XML schema provides a definition of a category of XML documents. However, developing a schema is of limited value unless there is some mechanical way to determine whether a given XML instance document conforms to the schema. Several XML schema validation tools are available. One of these is named `xsv`, an acronym for XML Schema Validator. It was developed by Henry S. Thompson and Richard Tobin at the University of Edinburgh in Scotland. If your schema and instance document are available on the Web, `xsv` can be used online, like the XHTML validation tool at the W3C Web site. This tool can also be downloaded and run on your computer. The Web site for `xsv` is `http://www.w3.org/XML/Schema#XSV`.

The output of `xsv` is an XML document. When run from the command line, the output document appears on the screen with no formatting, so it is a bit difficult to read. The following is the output of `xsv` when run on `planes.xml`:

```
<?XML version='1.0' encoding = 'utf-8'?>
<xsv docElt='{http://cs.uccs.edu/planeSchema}planes'
     instanceAssessed='true'
     instanceErrors = '0'
     rootType='[Anonymous]'
     schemaErrors='0'
     schemaLocs='http://cs.uccs.edu/planeSchema -> planes.xsd'
     target='file:/c:/wbook2/xml/planes.xml'
     validation='strict'
     version='XSV 1.197/1.101 of 2001/07/07 12:10:19'
     xmlns='http://www.w3.org/2000/05/xsv' >

    <importAttempt URI='file:/c:wbook2/xml/planes.xsd'
                   namespace='http://cs.uccs.edu/planeSchema'
                   outcome='success' />
</xsv>
```

The actual output was displayed with no formatting: Each line was filled to the right end of the screen, and attribute values were broken across line boundaries in several places.

One useful thing to know about validation with `xsv`: If the schema is not in the correct format, the validator will report that it could not find the specified schema.

7.7 Displaying Raw XML Documents

An XML-enabled browser, or any other system that can deal with XML documents, cannot possibly know how to format the tags defined in the document (after all, someone just made them up). Therefore, if you display an XML document without a style sheet that defines presentation styles for the document's tags, you should not expect it to have formatted content. Contemporary browsers include default style sheets that are used when no style sheet is specified in the XML document. The display of such an XML document is only a somewhat stylized listing of the XML. The FX2 browser display of the `planes.xml` document is shown in Figure 7.2.

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
- <!--
     planes.xml - A document that lists ads for
          used airplanes
  -->
- <!--
    <?xml-stylesheet  type = "text/css"  href = "planes.css" ?>
  -->
- <planes_for_sale>
  - <ad>
      <year> 1977 </year>
      <make> cessna </make>
      <model> Skyhawk </model>
      <color> Light blue and white </color>
    - <description>
        New paint, nearly new interior, 685 hours SMOH, full IFR King avionics
      </description>
      <seller phone="555-222-3333"> Skyway Aircraft </seller>
    - <location>
        <city> Rapid City, </city>
        <state> South Dakota </state>
      </location>
    </ad>
  - <ad>
      <year> 1965 </year>
      <make> Piper </make>
      <model> Cherokee </model>
      <color> Gold </color>
    - <description>
        240 hours SMOH, dual NAVCOMs, DME, new Cleveland brakes, great shape
      </description>
      <seller phone="555-333-2222"> John Seller </seller>
    - <location>
        <city> St. Joseph, </city>
        <state> Missouri </state>
      </location>
    </ad>
</planes_for_sale>
```

Figure 7.2 A display of an XML document with the FX2 default style sheet

Some of the elements in the display shown in Figure 7.2 are preceded by dashes. These elements can be elided (temporarily removed) by placing the mouse cursor over the dash and clicking the left mouse button. For example, if the mouse cursor is placed over the dash to the left of the first `<ad>` tag and the left mouse button is clicked, the result is as shown in Figure 7.3.

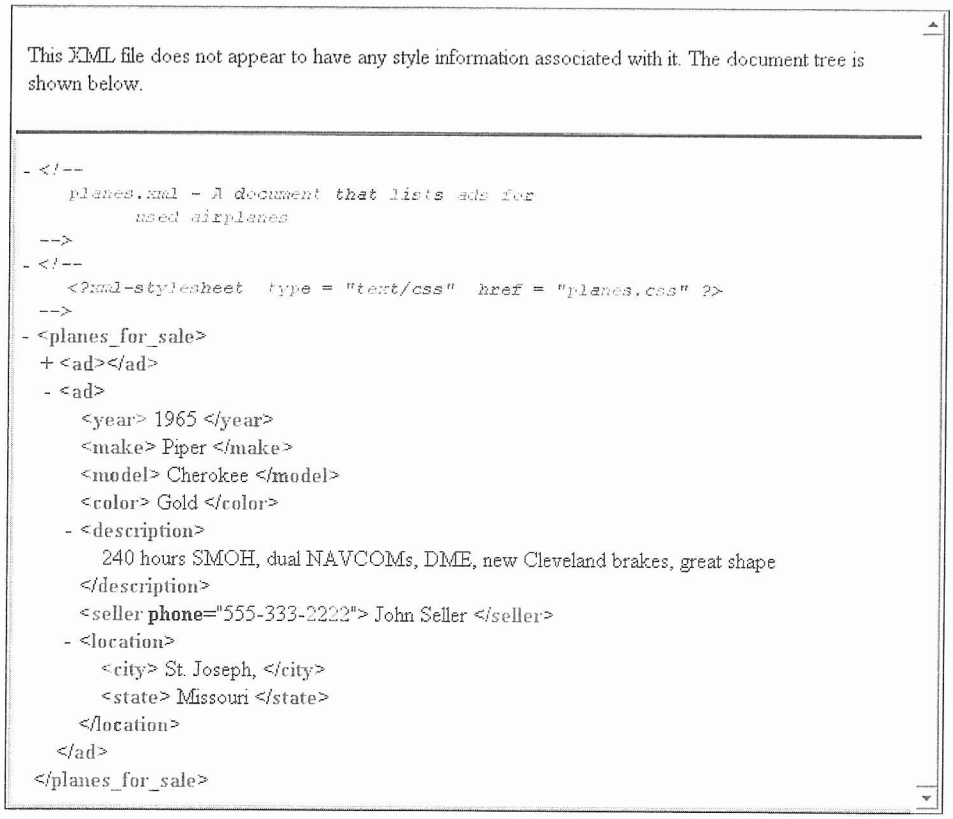

Figure 7.3 The document of Figure 7.2 with the first `ad` element elided

It is unusual to display a raw XML document. This is usually done to review and check the structure and content of the document during its development.

7.8 Displaying XML Documents with CSS

Style sheet information can be provided to the browser for an XML document in two ways. First, a Cascading Style Sheet (CSS) file that has style information for the elements in the XML document can be developed. Second, the XSLT style sheet technology, which was developed by the W3C, can be used. Although using CSS is effective, XSLT provides far more power over the appearance of the document's display. On the other hand, XSLT is not yet available on all of the most commonly used browsers. XSLT is discussed in Section 7.9.

The form of a CSS style sheet for an XML document is simple: It is just a list of element names, each followed by a brace-delimited set of the element's

CSS attributes. This is the form of the rules in a CSS document style sheet. The following shows a CSS style sheet for the `planes` XML document:

```
<!-- planes.css - a style sheet for the planes.xml document -->
ad { display: block; margin-top: 15px; color: blue;}
year, make, model { color: red; font-size: 16pt;}
color {display: block; margin-left: 20px; font-size: 12pt;}
description {display: block; margin-left: 20px; font-size: 12pt;}
seller { display: block; margin-left: 15px; font-size: 14pt;}
location {display: block; margin-left: 40px; }
city {font-size: 12pt;}
state {font-size: 12pt;}
```

The only style property in this style sheet that has not been discussed earlier in this book is `display`, which is used to specify whether an element is to be displayed inline or in a separate block. These two options are specified with the values `inline` and `block`. The `inline` value is the default. When `display` is set to `block`, the content of the element is usually separated from its sibling elements by line breaks.

The connection of an XML document to a CSS style sheet is established with the processing instruction `xml-stylesheet`, which specifies the particular type of the style sheet via its `type` attribute and the name of the file that stores the style sheet via its `href` attribute. For the `planes` example, this processing instruction is as follows:

```
<?xml-stylesheet type = "text/css" href = "planes.css" ?>
```

Figure 7.4 shows the display of `planes.xml` using the `planes.css` style sheet.

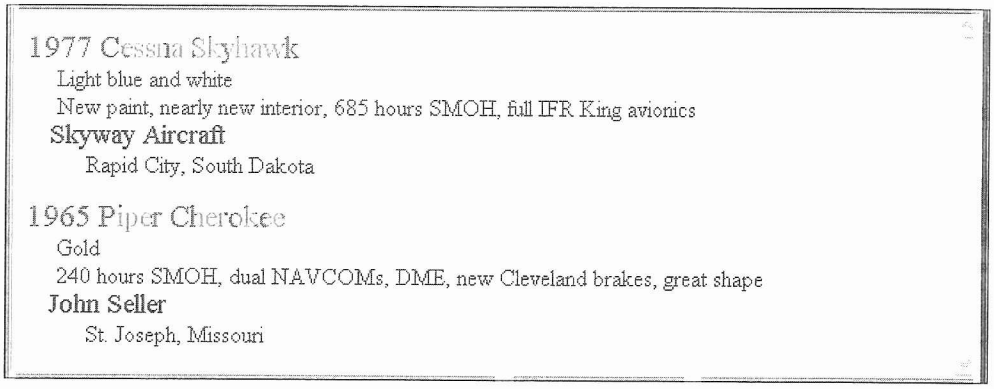

Figure 7.4 The result of using a CSS style sheet to format `planes.xml`

7.9 XSLT Style Sheets

The eXtensible Stylesheet Language (XSL) is a family of recommendations for defining XML document transformations and presentation. It consists of three related standards: XSL Transformations (XSLT), XML Path Language (XPath), and XSL Formatting Objects (XSL-FO). Each of these has an importance and use of its own. Together, they provide a powerful means of formatting XML documents. Because XSL-FO is not yet widely used, we do not discuss it in this book.

XSLT style sheets are used to transform XML documents into different forms or formats, perhaps using different DTDs. One common use for XSLT is to transform XML documents into XHTML documents, primarily for display. In the transformation of an XML document, element content can be moved and/or modified, sorted, or converted to attribute values, among other things. XSLT style sheets are XML doucments, so they can be validated against DTDs. They can even be transformed using other XSLT style sheets. The XSLT standard is given at `http://www.w3.org/TR/xslt`. XSLT stylesheets and their uses are the primary topics of this section.

XPath is a language for expressions, which are often used to identify parts of XML documents, such as specific elements that are in specific positions in the document or elements that have particular attribute values. XSLT requires such expressions to specify tranformations. XPath is also used for XML document querying languages, such as XQL, and for building new XML document structures using XPointer. The XPath standard is given at `http://www.w3.org/TR/xpath`. This chapter uses simple XPath expressions in the discussion of XSLT but does not explore them further.

7.9.1 Overview of XSLT

XSLT is actually a functional-style programming language. Included in XSLT are functions, parameters, names to which values can be bound, selection constructs, and conditional expressions for multiple selection. The syntactic structure of XSLT is XML, so each statement is specified with an element. This makes XSLT documents appear very different from programs in a typical programming language, but not completely different from programs written in the LISP-based functional languages COMMON LISP and Scheme.

XSLT processors take as input an XML document and an XSLT document. The XSLT document is the program to be executed; the XML document is the input data to the program. Parts of the XML document are selected, possibly modified, and merged with parts of the XSLT document to form a new document, which is sometimes called an *XSL document*. Note that the XSL document is also an XML document, which could be again the input to an XSLT processor. The output document can be stored for future use by applications, or it may be immediately displayed by an application, often a browser. Neither the XSLT document nor the input XML document is changed by the XSLT processor.

The transformation process by an XSLT processor is shown in Figure 7.5.

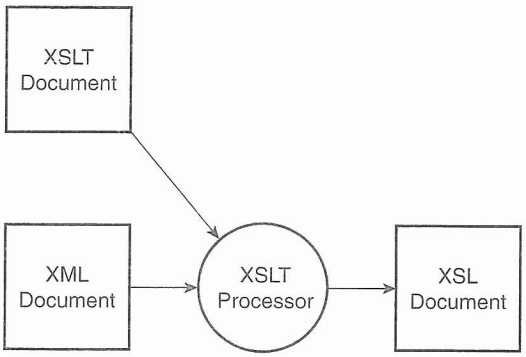

Figure 7.5 XSLT processing

An XSLT document consists primarily of one or more templates, which use XPath to describe element/attribute patterns in the input XML document. Each template has associated with it a section of XSLT "code," which is "executed" when a match to the template is found in the XML document. So, each template describes a function, which is executed whenever the XSLT processor finds a match to the template's pattern.

An XSLT processor sequentially examines the input XML document, searching for parts that match one of the templates in the XSLT document. XML documents consist of nodes, where nodes are elements, attributes, comments, text, and processing instructions. If a template matches an element, the element is not processed until the closing tag is found. When a template matches an element, the child elements of that element may or may not be processed.

The XSLT model of processing XML data is called the *template-driven model*, which works well when the data consists of multiple instances of highly regular data collections, as with files of records. XSLT can also deal with irregular and recursive data, using template fragments in what is called the *data-driven model*. A single XSLT style sheet can include the mechanisms for both the template- and data-driven models. The discussion of XSLT in this chapter is restricted to the template-driven model.

To keep the complexity of our discussion manageable, we focus on tranformations that are related to presentation. The examples in this section were processed with the XSLT processor that is part of IE7.

7.9.2 XSL Transformations for Presentation

Although XSLT style sheets can be used to control page layout, including orientation, writing direction, margins, and page numbering, this chapter discusses

only the simplest of formatting specifications for the smallest units of information. XSLT includes more than 50 formatting object (element) types and more than 230 properties, so it is a large and complex tag set.

In this section we assume that the XSLT processor processes an XML document with its associated XSLT style sheet document and produces as its output document an XHTML document to display.

An XML document that is to be used as data to an XSLT style sheet must include a processing instruction to inform the XSLT processor that the style sheet is to be used. The form of this instruction is as follows:

```
<?xml-stylesheet type = "text/xsl" href =
                                    "XSL_stylesheet_name" ?>
```

As a simple example of an XML document that can be used to illustrate XSLT formatting, consider the following:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!-- xslplane.xml -->
<?xml-stylesheet type = "text/xsl"  href = "xslplane.xsl" ?>
<plane>
   <year> 1977 </year>
   <make> Cessna </make>
   <model> Skyhawk </model>
   <color> Light blue and white </color>
</plane>
```

Notice that this document specifies `xslplane.xsl` as its XSLT style sheet.

An XSLT style sheet is an XML document whose root element is the special-purpose element `stylesheet`. The `stylesheet` tag defines namespaces as its attributes and encloses the collection of elements that defines its transformations. It also identifies the document as an XSLT document. The namespace for all XSLT elements is specified with a W3C URI. If the stylesheet includes XHTML elements, the stylesheet tag also specifies the XHTML namespace. Consider the following stylesheet tag:

```
<xsl:stylesheet xmlns:xsl =
                    "http://www.w3.org/1999/XSL/Format"
                xmlns = "http://www.w3.org/1999/xhtml">
```

Notice that the prefix for XSLT elements is `xsl` and the default namespace is that for XHTML.

A style sheet document must include at least one `template` element. The template opening tag includes a `match` attribute to specify an XPath expression to select a node in the XML document. The content of a template element specifies what is to be placed in the output document. If a template element is thought of as a subprogram, the opening tag states where the sub-

program is to be applied, and the content of the element specifies the body of the subprogram.

In most XSLT documents, a template is included to match the root node of the XML document. This can be done in two ways, one being to use the XPath expression `"/"`, as in the following:

```
<xsl:template match = "/">
```

Notation similar to that used to specify UNIX directory addresses is used. The alternative to using `"/"` is to use the actual root of the document. In the example, `xslplane.xml`, the document root is `plane`. Every XSLT style sheet should include a template for the root node. If the output of the XSLT processor is an XHTML document, the template that matches the root node is used to create the XHTML header of the output document. The header code appears as the content of the template element. An example of a complete template element follows:

```
<xsl:template match = "plane">
<html><head><title> Example </title></head><body>
...
</body></html>
</xsl:template>
```

To produce complete XHTML documents as output from XSLT documents, the `output` element can be included prior to the first template. This element can include `doctype-public` and `doctype-system` attributes to specify the two parts of the `DOCTYPE` declaration, respectively. For the sake of brevity, `output` elements are not included in the XSLT examples in this chapter.

Style sheets nearly always have templates for specific nodes of the XML document, which are descendants of the root node, as in the following example:

```
<xsl:template match = "year">
```

XPath expressions that begin with the slash are absolute addresses within the document. Those that do not begin with a slash are relative addresses. The value `"year"` in the preceding example is obviously a relative address. Relative addresses are relative to the "current" node of the XML document, which is the last node found by the XSLT processor in the XML document.

The template for the root node is implicitly applied. However, all other templates in an XSLT document must be explicitly applied to the XML document. This can be done in several ways. The `apply-templates` element applies appropriate templates to the descendant nodes of the current node. This element can include a `select` attribute to specify the descendant nodes whose templates should be applied. If no `select` attribute is included, the XSLT processor will apply a template to every descendant node. For those nodes for which the XSLT document has not defined a template, a default template is used. For example, both text and attributes have default templates that output them as text.

Template elements include two distinct kinds of elements: those that literally contain content and those that specify content to be copied from the associated XML document. XSLT elements that represent XHTML elements often are used to specify content. These have the appearance of their associated XHTML elements. For example, consider the following XHTML element:

```
<span style = "font-size: 14pt"> Merry Christmas! </span>
```

All XSLT elements that represent XHTML elements are copied by the XSLT processor to the output document being generated. Note that all XHTML elements that appear in an XSLT document must conform to the syntactic restrictions that apply to XML (and XHTML) elements.

In many cases, the content of an element of the XML document is to be copied to the output document. This is done with the `value-of` element, which uses a `select` attribute to specify the element of the XML document whose contents are to be copied. For example:

```
<xsl:value-of select = "AUTHOR" />
```

This element specifies that the content of the `AUTHOR` element of the XML document is to be copied to the output document. Because the `value-of` element cannot have content, it is terminated with a slash and a right angle bracket.

The `select` attribute can specify any node of the XML document. This is an advantage of XSLT formatting over CSS, in which the order of data as stored is the only possible order of display.

The attribute value `"."` for the `select` attribute of `value-of` means to select all elements within the current element, just the current node if it contains no nested elements.[3]

The following is a complete XSLT style sheet for the XML document `xslplane.xml`, shown previously:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!-- xslplane1.xsl
     An XSLT stylesheet for xslplane.xml using child templates
     -->
<xsl:stylesheet version = "1.0"
                xmlns:xsl = "http://www.w3.org/1999/XSL/Transform"
                xmlns = "http://www.w3.org/1999/xhtml">

<!-- The template for the whole document (the plane element) -->
   <xsl:template match = "plane">
     <html><head><title> Style sheet for xslplane.xml </title>
     </head><body>
```

3. If `select = "."` is included in an `<xsl:apply-templates>` tag, it does nothing because `apply-templates` implicitly specifies all immediate child nodes.

```
      <h2> Airplane Description </h2>

<!-- Apply the matching templates to the elements in plane -->
      <xsl:apply-templates />
      </body></html>
    </xsl:template>

<!-- The templates to be applied (by apply-templates) to the
      elements in the plane element -->
    <xsl:template match = "year">
      <span style = "font-style: italic; color: blue;"> Year:
      </span>
      <xsl:value-of select = "." /> <br />
    </xsl:template>
    <xsl:template match = "make">
      <span style = "font-style: italic; color: blue;"> Make:
      </span>
      <xsl:value-of select = "." /> <br />
    </xsl:template>
    <xsl:template match = "model">
      <span style = "font-style: italic; color: blue;"> Model:
      </span>
      <xsl:value-of select = "." /> <br />
    </xsl:template>
    <xsl:template match = "color">
      <span style = "font-style: italic; color: blue;"> Color:
      </span>
      <xsl:value-of select = "." /> <br />
    </xsl:template>
</xsl:stylesheet>
```

Figure 7.6 shows an IE7 display of the output document created by the XSLT processor from `xslplane.xml` with `xslplane1.xsl`.

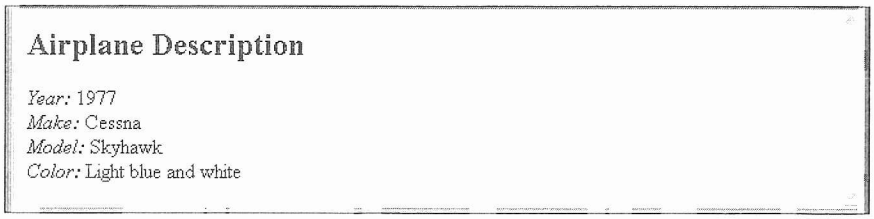

Figure 7.6 An output document from the XSLT processor

The XSLT document, `xslplane1.xsl`, is more general and complex than necessary for the simple use for which it was written. There is actually no need to include templates for all of the child nodes of `plane`, because the `select` clause of the `value-of` element finds them. The following XSLT document, `xslplane2.xsl`, produces the same output as `xslplane1.xsl`.

```
<?xml version = "1.0" encoding = "utf-8"?>
<!-- xslplane2.xsl
     An XSLT Stylesheet for xslplane.xml using implicit templates
     -->
<xsl:stylesheet version = "1.0"
                xmlns:xsl = "http://www.w3.org/1999/XSL/Transform"
                xmlns = "http://www.w3.org/1999/xhtml">

<!-- The template for the whole document (the plane element) -->
     <xsl:template match = "/">
     <html><head><title> Style sheet for xslplane.xml </title>
     </head><body>
     <h2> Airplane Description </h2>
     <span style = "font-style: italic; color: blue;"> Year:
     </span>
     <xsl:value-of select = "year" /> <br />
     <span style = "font-style: italic; color: blue;"> Make:
     </span>
     <xsl:value-of select = "make" /> <br />
     <span style = "font-style: italic; color: blue;"> Model:
     </span>
     <xsl:value-of select = "model" /> <br />
     <span style = "font-style: italic; color: blue;"> Color:
     </span>
     <xsl:value-of select = "color" /> <br />
     </body></html>
   </xsl:template>
</xsl:stylesheet>
```

We now consider an XML document that includes a collection of data elements with the same structure. For example, a document named `airplanes.xml` could have a list of airplane descriptions. The XSLT template used for one plane can be used repeatedly with the `for-each` element, which uses a `select` attribute to specify an element in the XML data. The value of the `select` attribute is a pattern, which is a path expression that specifies an element. Any child elements of the specified element are included.

Consider the following XML document:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!-- xslplanes.xml -->
<?xml-stylesheet type = "text/xsl" href = "xslplanes.xsl" ?>
<planes>
   <plane>
      <year> 1977 </year>
      <make> Cessna </make>
      <model> Skyhawk </model>
      <color> Light blue and white </color>
   </plane>
   <plane>
      <year> 1975 </year>
      <make> Piper </make>
      <model> Apache </model>
      <color> White </color>
   </plane>
   <plane>
      <year> 1960 </year>
      <make> Cessna </make>
      <model> Centurian </model>
      <color> Yellow and white </color>
   </plane>
   <plane>
      <year> 1956 </year>
      <make> Piper </make>
      <model> Tripacer </model>
      <color> Blue </color>
   </plane>
</planes>
```

The following XSLT style sheet processes the previous XML data document:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!-- xslplanes.xsl -->
<xsl:stylesheet version = "1.0"
                xmlns:xsl = "http://www.w3.org/1999/XSL/Transform"
                xmlns = "http://www.w3.org/1999/xhtml" >

<!-- The template for the whole document (the planes element) -->
   <xsl:template match = "planes">
      <h2> Airplane Descriptions </h2>
```

```
<!-- Apply the following to all occurrences of the plane element -->
      <xsl:for-each select = "plane">
         <span style = "font-style: italic"> Year: </span>
         <xsl:value-of select = "year" /> <br />
         <span style = "font-style: italic"> Make: </span>
         <xsl:value-of select = "make" /> <br />
         <span style = "font-style: italic"> Model: </span>
         <xsl:value-of select = "model" /> <br />
         <span style = "font-style: italic"> Color: </span>
         <xsl:value-of select = "color" /> <br /> <br />
      </xsl:for-each>

   </xsl:template>
</xsl:stylesheet>
```

Figure 7.7 shows an IE7 display of the document produced by an XSLT processor on `xslplanes.xml`, using the `xslplanes.xsl` style sheet.

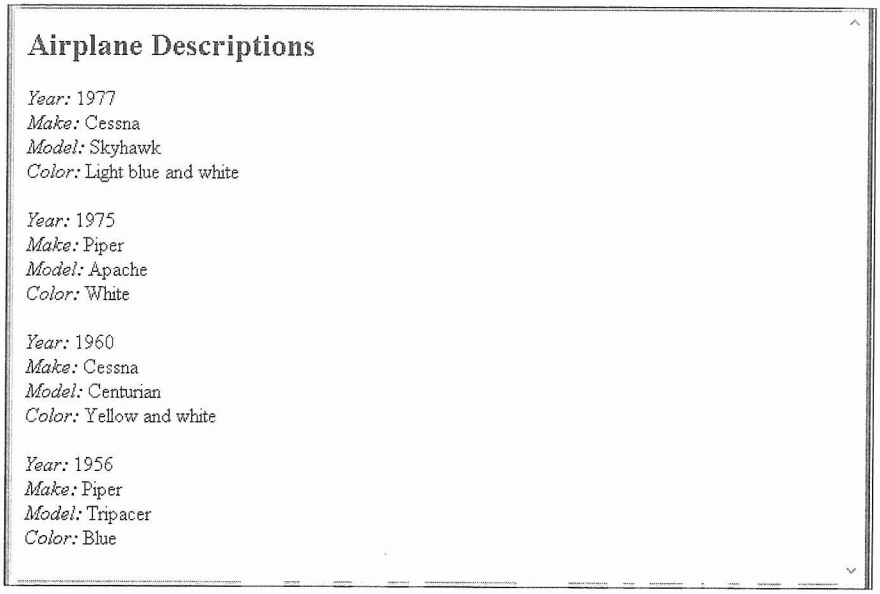

Figure 7.7 Using the `for-each` element for lists of elements

XSLT provides a simple way to sort the elements of the XML document before sending them or their content to the output document. This is done with the `sort` element, which can take several attributes. The `select` attribute

specifies the node that is used for the key of the sort. The `data-type` attribute is used to specify whether the key is to be sorted as text (`"text"`) or numerically (`"number"`). By default, the sort is to ascending order. The `order` attribute can be set to `"descending"` to produce the reverse order. By inserting the following single line into the `xslplanes.xsl` document, the output will appear in ascending numeric order of the year of the airplane:

```
<xsl:sort  select = "year"  data-type = "number" />
```

7.10 XML Processors

So far this chapter has discussed the structure of XML documents, the rules for writing them, the DTD and XML schema approaches to specifying the particular tag sets and structure of collections of XML documents, and the CSS and XSLT methods of displaying the contents of XML documents. That is tantamount to telling a long story about how data can be stored and displayed, without providing any hint on how it may be processed. Although this section does not discuss processing data stored in XML documents, it does introduce approaches to making that data conveniently available to application programs that process that data.

7.10.1 The Purposes of XML Processors

Several purposes of XML processors have already been discussed. They are as follows: First, the processor must check the basic syntax of the document for well-formedness. Second, the processor must replace all references to entities in an XML document by their definitions. Third, DTDs and XML schemas can specify that certain values in an XML document have default values, which must be copied into the XML document during processing. Fourth, when a DTD or an XML schema is specified and the processor includes a validating parser, the structure of the XML document must be checked to ensure that its structure is legitimate.

One simple way to check the well-formedness of an XML document is with a browser that has an XML parser. Another way is to run an XML parser directly on the document. One such parser is available from the Apache XML Project Xerces-J. It can be downloaded from `http://xml.apache.org/xerces-j/index.hmtl`. This parser is written in Java and runs on any platform that has a Java virtual machine Version 1.1 or later. The class from this system that checks the well-formedness of XML documents, which can be run from the command line, is named `sax.SAXCount`.[4]

Although an XML document exhibits a regular and elegant structure, that structure does not provide applications convenient access to the document's data. Because the process of the initial syntactic analysis required to expose the embedded data must be repeated for every application that processes XML

4. The term SAX here is borrowed from the acronym SAX, which is derived from Simple Application Programming Interface for XML.

documents, it was recognized early on that standard syntax analyzers for XML documents were needed. Actually, the syntax analyzers themselves need not be standard; rather, they should expose the data of XML documents in a standard application programmer interface (API). This need led to the development of two different standard APIs for XML processors. Because there are different needs and uses of XML applications, having two standards is not a negative. The two APIs parallel the two kinds of output that are produced by the syntax analyzers of compilers for programming languages. Some of these syntax analyzers produce a stream of the syntactic structures of an input program. Others produce a parse tree of the input program that shows the hierarchical structure of the program in terms of its syntactic structures.

7.10.2 The SAX Approach

The Simple API for XML (SAX) standard, which was released in May 1998, was developed by an XML users group, XML-DEV. Although not developed or supported by any standards organization, SAX has been widely accepted as a de facto standard and is now widely supported by XML processors.

The SAX approach to processing is called *event processing*. The processor scans the XML document from beginning to end. Every time a syntactic structure of the document is recognized, the processor signals an event to the application by calling an event handler for the particular structure that was found. The syntactic structures of interest naturally include opening tags, attributes, text, and closing tags. The interfaces that describe the event handlers form the SAX API.

7.10.3 The DOM Approach

The natural alternative to the SAX approach to XML document parsing is to build a hierarchical syntactic structure of the document. Given the use of DOM representations of XHTML documents to create dynamic documents in Chapter 6, "Dynamic Documents with JavaScript," this is a familiar idea. In the case of XHTML, the browser parses the document and builds the DOM tree. In the case of XML, the parser part of the XML processor builds the DOM tree. In both cases, the nodes of the tree are represented as objects that can be accessed and processed or modified by the application. When parsing is complete, the complete DOM representation of the document is in memory and can be accessed in a number of different ways, including tree traversals of various kinds as well as random accesses.

The DOM representation of an XML document has several advantages over the sequential listing provided by SAX parsers. First, it has an obvious advantage if any part of the document must be accessed more than once by the application. Second, if the application must perform any rearrangement of the document, that can most easily be done if the whole document is accessible at the same time. Third, accesses to random parts of the document are possible. Finally, because the parser sees the whole document before any processing takes

place, this approach avoids any processing of a document that is later found to be invalid (according to a DTD or XML schema).

In some situations, the SAX approach has advantages over the DOM method. The DOM structure is stored entirely in memory. For large documents, this requires a great deal of memory. In fact, because there is no limit on the size of an XML document, there may be some documents that cannot be parsed this way. This is not a problem with the SAX approach. Another advantage of the SAX method is speed—it is faster than the DOM approach.

The process of building the DOM structure of an XML document requires some syntactic analysis of the document, similar to that done by SAX parsers. In fact, most DOM parsers include a SAX parser as a front end.

7.11 Web Services

The movement toward Web services began in earnest when Microsoft Chairman Bill Gates introduced a concept he called BizTalk in 1999. BizTalk later was renamed .NET. The idea was to provide the technologies to allow different software in different places, written in different languages and resident on different platforms, to connect and interoperate.

The Web began as a focused Web service for information, and is still just that. Through two fundamental methods, `GET` and `POST`, and a vast collection of public markup documents, information is provided to anyone with an Internet connection and a computer running a browser. The more general concept of a Web service is a similar technology for services. Rather than deploying documents through a Web server, services are deployed (through the same Web server). Rather than documents, access to software components is provided. Components are not downloaded, but are run on the Web server as a remote service.

Web services are of course not a completely new idea. Remote Procedure Call (RPC) is an earlier and closely related concept. RPC was invented to allow distributed components to communicate. There are two successful (widely used) RPC technologies, DCOM and CORBA. Both, however, are too complex to provide a simple and convenient way to support interoperability among the components of different systems. DCOM is proprietary, supported only by Microsoft software systems. CORBA is designed to be cross-platform, but it requires a great deal of manual integration work. DCOM uses the Object Remote Procedure Call (ORPC) protocol to interface components. CORBA uses Object Management Group's Internet Inter-ORB Protocol (IIOP). Needless to say, these two protocols are not compatible. Therefore, neither DCOM nor CORBA supports the goal of Web services—universal component interoperability.

The dream of Web services is that there will be protocols that allow all components to interoperate entirely under the control of the computers, without human intervention. This means that when a system needs a service, it can implicitly find one on the Web and use it. Standard non-proprietary protocols and languages to support this dream have been developed, although they are not yet widely used.

There are three roles that are required to provide and use Web services: service providers, service requestors, and a service registry. A service provider must develop and deploy software that provides a service. This service must have a standard description. The W3C language designed for writing such descriptions is Web Services Definition Language (WSDL, pronounced "wiz´-dul"), which is an XML-based format. The WSDL description is published on a Web server, similar to a Web-accessible document. It is used to describe the specific operations provided by the Web service, as well as the protocols for the messages the Web service can send and receive. The descriptions of data, both input and output, in a WSDL description are often written using XML schema.

A Web services registry is created with another standard protocol, Universal Description, Discovery and Integration Service (UDDI). UDDI also provides methods of querying a Web services registry to determine what specific services are available. So, a requestor queries a registry with a WSDL query, to which the registry responds with the protocol of how the requestor may interact with the requested Web service. UDDI has two kinds of clients, service providers and clients who want to find and use Web services.

SOAP is an XML tag set that defines the forms of messages and RPCs. SOAP was originally an acronym for Standard Object Access Protocol, designed to describe data objects. However, it is now a name for the XML tag set with wider use in Web services communications. The root element of a SOAP document is `Envelope`, so SOAP documents are often called envelopes. The body of a SOAP message is either a request, which is an RPC, or a response, which contains values returned from the called method, or service. SOAP messages are sent with the HTTP POST method.

There are now tools for creating Web services that construct SOAP messages, so that one can build Web services without directly writing any SOAP documents. The Java Web Services Developer Pack is one such tool.

Chapter 12, "Introduction to ASP.NET," discusses Web services provided through ASP.NET.

Summary

XML is a simplified version of SGML, which is a meta-markup language. XML provides a standard way for a group of users to define the structure of their data documents, using a subject-specific markup language.

XML documents can include elements, markup declarations, and processing instructions. Every XML document has the form of a single document tree, so there can be just one root element.

An XML document is a document entity that can include any number of references to other entities defined elsewhere. An entity can be several different things, including plain text and references to images.

A DTD is a document that describes the syntactic structure of an XML document or collection of documents that uses a particular tag set. A validating XML parser compares a document it is analyzing to its DTD, if one is specified.

If no DTD is specified for an XML document, only well-formedness can be checked during parsing. A DTD has declarations for elements, attributes, entities, and notations. An element declaration specifies the name of the element and its structure. If an element represents an internal node in the document tree, its structure is a list of the children nodes. Any internal node can include a modifier that specifies the number of times that its children nodes can or must appear. A leaf node's structure is usually either empty or plain text.

A DTD attribute declaration specifies the attribute's name, the name of its associated element, the type of its values, and optionally, a default value. In many cases, the type of an attribute value is simply text. The default value can be an actual value, but it may also specify something about the value. There are several predefined entities that represent the special characters that are used as markup delimiters. A character data section can be used to allow these special characters to appear as themselves, without using entities. A DTD specification could appear embedded in an XML document, but this makes it inconvenient to use for other documents.

An XML document can include the predefined element names for some other application, such as the names of the elements of XHTML. To avoid name clashes between these different sources of names, XML uses the concepts of namespaces and name prefixes, which indicate the namespace of a name in a document. Namespaces are specified in declarations as URIs. A default namespace can be declared for a document. Names from the default namespace can be used without being prefixed.

XML schemas provide an alternative to DTDs. XML schemas allow much stricter control over the structure and especially the data types of an XML document. A schema defines the structure of a class of XML documents. The documents that conform to a specific schema are considered instances of that schema. A schema, which is an XML document, is an instance of a schema, `XMLSchema`. A schema specifies a target namespace with the `targetNamespace` attribute. The target namespace is also often named as the default namespace. Schemas can define simple and complex data types. Simple data types cannot contain other elements or attributes. One common category of complex types are those that can contain other elements but no text. There are many predefined types. Users are allowed to define new simple types as constrained versions of existing simple types, using facets. Users can also define new complex types. Instances of schemas can be validated with several different validation programs that are now available, among them `xsv`.

An XML parser includes a default style sheet, which is used when no other style sheet is specified by the document being parsed. The default style sheet simply produces a somewhat stylized listing of the XML. CSS style sheets can be used with XML documents to provide formatting information. Such a CSS style sheet has the form of an external CSS style sheet for XHTML.

XML documents can also be formatted with XSLT style sheets. XSLT style sheets specify document transformations and can include XHTML and CSS presentation information. XSLT style sheets define templates into which XML document elements are mapped. An XSLT processor creates an output docu-

ment from the XML document and the XSLT style sheet. If the style sheet includes XHTML style specifications, the document will have style information embedded in its elements. XSLT style sheets actually are XML applications. An XSLT style sheet can have a template that is reused for any number of occurrences of a document branch in the associated XML document.

XML applications require that the nodes (tags, attributes, text, etc.) of the XML document are provided in some standard way by the XML parser. The two ways in which this is done are the SAX approach, which calls an event handler for each node it finds, and the DOM approach, which provides a complete tree structure of the whole document.

A Web service is a method that resides and is executed on a Web server, but which can be called from any computer on the Web. The standard technologies to support Web services are WSDL, UDDI, SOAP, and XML.

Review Questions

7.1 Is XML more closely related to SGML or HTML?

7.2 What is the main deficiency of HTML?

7.3 What is the goal of HTML?

7.4 What is the goal of XML?

7.5 What are the two primary tasks of a validating XML parser?

7.6 In what way are nested tags better than attributes?

7.7 What is a document entity?

7.8 Why should a document be broken into multiple entities?

7.9 What is a binary entity?

7.10 How does an XML parser handle binary entities?

7.11 What is the purpose of a DTD?

7.12 Why is it better to find an error in a DTD before it is used?

7.13 What are the four possible keywords in a DTD declaration?

7.14 What are the meanings of the modifiers (+, *, and ?) that can be used in element declarations?

7.15 Explain the three types that can be used to describe data in an element declaration.

7.16 What are the four possible parts of an attribute declaration in a DTD?

7.17 Describe the meanings of the default attribute values `#REQUIRED` and `#IMPLIED`.

7.18 What is the difference between general and parameter entities?

7.19 Why do some special characters have predefined entity references?

7.20 What is the purpose of a character data section?

7.21 How does the XML parser distinguish between a general entity and a parameter entity?

7.22 What does the keyword `SYSTEM` specify in an entity declaration?

7.23 What is the syntactic form of an internal DTD?

7.24 What is the markup vocabulary of a markup language?

7.25 What is an XML namespace?

7.26 What are the two primary advantages of XML schemas over DTDs?

7.27 From where do the names used in defining an XML schema come?

7.28 What three namespaces are normally named in an XML schema?

7.29 What is the form of the assignment to the `schemaLocation` attribute?

7.30 What are the differences between simple and complex XML schema types?

7.31 Define local and global declarations in an XML schema.

7.32 What is a facet?

7.33 What are the four categories of complex types in an XML schema?

7.34 What is the difference between the `sequence` and `all` schema elements?

7.35 Why would you use a CSS style sheet for an XML document?

7.36 How does an XSLT processor use an XSLT style sheet with an XML document?

7.37 What is a `template` element of an XSLT style sheet?

7.38 What two kinds of elements are included in XSLT style sheets?

7.39 What does the `value-of` XSLT element do?

7.40 What does the `select` attribute of the `value-of` element do?

7.41 What does the `for-each` element of an XSLT style sheet do?

7.42 What is produced by a SAX parser?

7.43 What is produced by a DOM parser?

7.44 What advantages does a SAX parser have over a DOM parser?

7.45 What advantages does a DOM parser have over a SAX parser?

7.46 Explain the ultimate goal of Web services.

7.47 Describe the three roles in Web services.

7.48 What is UDDI?

7.49 What is SOAP?

Exercises

Write, test, and debug (if necessary) the following documents.

7.1 Create a DTD for a catalog of cars, where each `car` has the child elements `make`, `model`, `year`, `color`, `engine`, `number_of_doors`, `transmission_type`, and `accessories`. The `engine` element has the child elements `number_of_cylinders` and `fuel_system` (carbureted or fuel-injected). The `accessories` element has the attributes `radio`, `air_conditioning`, `power_windows`, `power_steering`, and `power_brakes`, each of which is required and has the possible values `yes` and `no`. Entities must be declared for the names of popular car makes.

7.2 Create an XML document with at least three instances of the `car` element defined in the DTD of Exercise 7.1. Process this document using the DTD of Exercise 7.1 and produce a display of the raw XML document.

7.3 Create an XML schema for the XML document described in Exercises 7.1 and 7.2.

7.4 Create a CSS style sheet for the XML document of Exercise 7.2 and use it to create a display of that document.

7.5 Create an XSLT style sheet for one `car` element of the XML document of Exercise 7.2 and use it to create a display of that element.

7.6 Modify the XSLT style sheet of Exercise 7.5 to format all the `car` elements in the XML document of Exercise 7.2 and use it to create a display of the whole document.

7.7 Design an XML document to store information about patients in a hospital. Information about patients must include name (in three parts), Social Security number, age, room number, primary insurance company—including member identification number, group number, phone number, and address—secondary insurance company (in the same subparts as for the primary insurance company), known medical problems, and known drug allergies. Both attributes and nested tags must be included. Make up sample data for at least four patients.